

QV-net: Decentralized Self-Tallying Quadratic Voting with Maximal Ballot Secrecy

Zibo Zhou
zb99.zhou@gmail.com
School of Cyber Science and
Technology, Beihang University
Beijing, China

Zongyang Zhang*
zongyangzhang@buaa.edu.cn
School of Cyber Science and
Technology, Beihang University
Beijing, China
Beijing Advanced Innovation Center
for Future Blockchain and Privacy
Computing
Beijing, China

Feng Hao
feng.hao@warwick.ac.uk
Department of Computer Science,
University of Warwick
Coventry, United Kingdom

Bowen Zheng
zhbowen04@gmail.com
School of Cyber Science and
Technology, Beihang University
Beijing, China

Zulkarnaim Masyhur
zulkarnaim.masyhur@warwick.ac.uk
Department of Computer Science,
University of Warwick
Coventry, United Kingdom

Abstract

Decentralized e-voting enables secure and transparent elections without relying on trusted authorities, with blockchain emerging as a popular platform. It has compelling applications in Decentralized Autonomous Organizations (DAOs), where governance relies on voting with blockchain-issued tokens. Quadratic voting (QV), a mechanism that mitigates the dominance of large token holders, has been adopted by many DAO elections to enhance fairness. However, current QV systems deployed in practice publish voters' choices in plaintext with digital signatures. The open nature of all ballots comprises voter privacy, potentially affecting voters' honest participation. Prior research proposes using cryptographic techniques to encrypt QV ballots, but they work in a centralized setting, relying on a trusted group of tallying authorities to administrate an election. However, in DAO voting, there is no trusted third party.

In this paper, we propose QV Network (QV-net), the first decentralized quadratic voting scheme, in which voters do not need to trust any third party other than themselves for ballot secrecy. QV-net is self-tallying with maximal ballot secrecy. Self-tallying allows anyone to compute election results once all ballots are cast. Maximal ballot secrecy ensures that what each voter learns from QV-net is nothing more than the tally and their own ballot. We provide an open-source implementation of QV-net to demonstrate its practicality based on real-world DAO voting settings, reporting only a few milliseconds for voting and a maximum of 255 milliseconds for tallying.

The exceptional efficiency of QV-net is attributed to the design of two new Zero-Knowledge Argument of Knowledge (ZKAoK) protocols for QV ballot secrecy and integrity. Previous works generally rely on pairing-friendly curves to prove the well-formedness of an encrypted QV ballot. But they incur heavy computation and large data sizes. We tackle the challenges of appropriately formalizing and proving ZKAoK relations for QV without using these curves. Specifically, we develop a succinct ZKAoK to prove a new

relation: the sum of squares of a private vector's components equals a private scalar. We also introduce the first aggregated range proof to prove that values committed under different keys fall within their respective ranges. Together, these two new zero-knowledge protocols enable us to build an efficient decentralized QV scheme and are of independent interest.

Keywords

Decentralized e-Voting; Quadratic Voting; DAOs; ZKAoKs

1 Introduction

Decentralized e-voting [27, 29, 34] allows voters to conduct and verify elections without relying on trusted authorities, ensuring transparency and integrity through distributed mechanisms. Recently, blockchain has become a prominent platform for such schemes due to its decentralized, transparent, and immutable nature [31]. Decentralized e-voting is particularly crucial for Decentralized Autonomous Organizations (DAOs), a new form of organization that enables self-governance through self-executing rules on an open platform such as a blockchain [30].

DAOs are collectively governed by members through decisions on proposals and economic rights via blockchain-issued governance tokens [18]. As of December 30th, 2024, DeepDAO¹ reports DAOs' treasury at 34 billion, with 2,459 organizations. Voting schemes are essential for DAOs' decision-making and healthy growth [21]. DAOs use various voting methods, including token-based quorum voting, quadratic voting, and holographic consensus voting, etc. Among these, quadratic voting is notable for addressing the "whale problem", where large token holders dominate governance [54].

Quadratic voting (QV), introduced by Lally and Weyl [36], is a social choice mechanism where the cost of votes increases quadratically with the number of votes. To cast n votes, a voter needs to spend n^2 tokens, which effectively prevents wealthy token holders from dominating the voting outcome while still allowing them

*Corresponding author.

¹<https://deepdao.io/organizations>

to express strong preferences. In DAOs, QV has been adopted by organizations such as MetFi DAO [14], Karmaverse [32], Pistachio DAO [15], Gitcoin [25], and MoonDAO [41].

However, existing QV implementations publish the plaintext votes together with digital signatures [14, 15, 25, 32, 41]. The open nature of ballots threatens voter privacy. It is possible to encrypt each QV vote under a joint public key and put the private key under the threshold control of a group of tallying authorities [43, 45]. However, this requires voters to trust an external group of people. If the tallying authorities receive a subpoena, they would have no interest in defying the order and going to jail to protect voters' privacy [34]. This is especially important in DAOs since a trusted third party does not exist there. Developing a QV scheme that achieves ballot secrecy without any trusted third party remains an open challenge. Furthermore, previous cryptographic QV designs [43, 45] rely on pairing-friendly curves to perform encryption and zero-knowledge proofs, but they incur high computation and large message sizes. Designing a verifiable QV scheme that can be implemented using standard elliptic curves without pairing is desirable and presents another challenge.

1.1 Our contributions

In this paper, we propose QV-net, a decentralized self-tallying quadratic voting scheme with maximal ballot secrecy. Self-tallying [34] allows anyone to compute the election result after all ballots are cast, while maximal ballot secrecy [34] ensures that the partial tally of the ballots of a set of voters is only accessible to the coalition of all remaining voters. We summarize our contributions as follows.

A decentralized self-tallying quadratic voting with maximal ballot secrecy. We propose the first decentralized quadratic voting achieving both self-tallying and maximal ballot secrecy. To this end, we adapt the distributed key generation algorithm from [29] and utilize Zero-Knowledge Arguments of Knowledge (ZKAoKs). Specifically, we formalize a series of ZKAoK relations for ballot secrecy and validity in decentralized QV, including the discrete logarithm knowledge relation, the discrete logarithm equality relation, the sum-of-square relation, and the range relation. For the first two relations, we apply classic Σ protocols. For the latter two relations, we develop two new ZKAoKs, which are of independent interest.

A succinct ZKAoK for a sum-of-square relation. We design a succinct ZKAoK to prove that the sum of squares of all components of a private vector equals a private scalar. For a vector of length n , it achieves $O(n)$ prover complexity, $O(\log n)$ proof size and $O(n)$ verifier complexity.

An aggregated range proof with different bases and ranges. We propose VarRange, an aggregated range proof supporting different bases and ranges. Given n Pedersen commitments with different commitment keys, VarRange proves that each committed value lies within its respective range. It achieves $O(nb)$ prover complexity, $O(\log nb)$ proof size and $O(nb)$ verifier complexity, where b is the maximum bit length among specified ranges.

A proof-of-concept implementation of QV-net. We present a proof-of-concept implementation for QV-net to assess its performance. The results demonstrate its practicality. Under parameter scales derived from real-world DAO proposals, including MetFi DAO, Karmaverse, Pistachio DAO, Gitcoin, and MoonDAO, QV-net

requires at most 170 ms for ballot casting and ZKAoK proof generation, 255 ms for tallying, 42 ms for ZKAoK proof verification, and 7.69 KB for the total data published during the voting process.

1.2 Related work

Decentralized e-voting. E-voting has been a longstanding research topic. Centralized e-voting systems employ trustworthy authorities to administrate the election process, but the authorities are difficult to find in practice and their worthiness is often called into question. Kiayias and Yung [34] introduced the concept of self-tallying in decentralized e-voting, allowing anyone to compute the election result once all ballots are cast. Follow-up works [27, 29] further improved the efficiency. However, these schemes are primarily designed for yes/no voting on a single choice.

Blockchain has become an enabling platform for implementing decentralized e-voting due to its transparent and immutable nature. McCorry et al. [40] presented the first blockchain-based implementation of a decentralized self-tallying voting scheme from [29]. Panja et al. [42] extended this implementation to support a ranked-choice voting, where voters rank choices by preference. Yang et al. [61] proposed a blockchain-based self-tallying scheme for score voting, where voters rate each choice with a score, and the choices with the highest total scores are elected. Li et al. [38] improved fairness in blockchain-based self-tallying voting for decentralized IoT, though they remain focused on yes/no voting.

Quadratic voting. Lalley and Weyl [36] introduced quadratic voting and demonstrated that a quadratic vote pricing rule is optimally robust. Weyl [59] showed that QV is relatively resistant to certain forms of collusion, fraud, and aggregate uncertainty. As summarized in [24], QV has been examined from ethical [37], practical [48, 49] and legal [47] perspectives, with applications including corporate governance [46], blockchain governance [17], civic engagement [2], and survey research [49].

To enhance privacy and integrity, Park and Rivest [43], Pereira and Rønne [45] proposed QV with end-to-end verifiability. However, they are limited to the centralized setting. Neither work [43, 45] provides an implementation. QV-net is the first decentralized voting scheme that supports QV (see Table 1 for a summary). QV-net uses standard elliptic curves without pairing, and we provide a full implementation to show its practicality (see §6).

The original QV focuses on binary decisions with real money payments. Eguia et al. [20] generalized QV to multiple-choice elections, while Posner and Stephanopoulos [47] introduced fixed-budget QV to overcome the problematic use of real money. To combine the benefits of both variations, Georgescu et al. [24] formally proposed fixed-budget multiple-choice QV.

In DAOs, QV has emerged as one of the top voting mechanisms in 2024², with prominent organizations such as MetFi DAO [14], Karmaverse [32], Pistachio DAO [15], Gitcoin [25], and MoonDAO [41] adopting it for decision-making on proposals.

Range proofs. Range proofs allow a prover to convince a verifier that a secret value belongs to a certain range. They are typically constructed via approaches including square decomposition, n -ary decomposition, hash chains, and generic zero-knowledge proofs. Kindly refer to [10] for a comprehensive SoK.

²<https://www.krayondigital.com/blog/top-dao-voting-mechanisms-compared-2024>

Table 1: Comparison of related e-voting schemes

	KY02 [34]	Gro04 [27]	OV-net [29]	MSH17 [40]	PBH ⁺ 20 [42]	PriScore [61]	LSY ⁺ 22 [38]	PR17 [43]	PR20 [45]	QV-net
Voting mechanism	yes/no	yes/no	yes/no	yes/no	ranked-choice	score	yes/no	quadratic	quadratic	quadratic
Setting	DEC	DEC	DEC	DEC	DEC	DEC	DEC	CEN	CEN	DEC

Notes: DEC denotes decentralized. CEN denotes centralized.

An aggregated range proof enables a single prover to generate a short proof for multiple range statements simultaneously. Several range proof protocols have this desirable property, including Bulletproofs [5], Bulletproofs+ [11], Bulletproofs++ [19], Flashproofs [57], and SwiftRange [58]. However, none of them supports aggregation across different commitment keys and ranges. Sharp [13] and MissileProof [23] are vector range proofs proving every element in a committed vector lies in some range. However, Sharp requires costly RSA or class groups while MissileProof requires expensive FFT (Fast Fourier Transform) operations.

2 Preliminaries

2.1 Notations

Let $\mathbb{G}$ denote a cyclic group of prime order p with generator g , written in multiplicative notation. Let $\mathbb{F}_p$ denote the finite field of order p . For $n \in \mathbb{N}$, let $[n]$ be the set $\{1, 2, \dots, n\}$, and $\mathbb{G}^n, \mathbb{F}_p^n$ denote vector space of dimension n over $\mathbb{G}, \mathbb{F}_p$, respectively.

We use bold letters to denote vectors. For $c \in \mathbb{F}_p, \mathbf{a} = (a_1, \dots, a_n) \in \mathbb{F}_p^n, \mathbf{g} = (g_1, \dots, g_n) \in \mathbb{G}^n$, we write $\mathbf{ca}$ for $(ca_1, \dots, ca_n)$, $\mathbf{g}^c$ for $(g_1^c, \dots, g_n^c)$, and $\mathbf{c}^n$ for $(c^0, c^1, \dots, c^{n-1})$. For vectors $\mathbf{a}, \mathbf{b} \in \mathbb{F}_p^n$, we write $\mathbf{a} \circ \mathbf{b} = (a_1b_1, \dots, a_nb_n)$ to denote the Hadamard product, $\langle \mathbf{a}, \mathbf{b} \rangle = \sum_{i=1}^n a_ib_i$ to denote the inner product. We write $\mathbf{g}^{\mathbf{a}} = \prod_{i=1}^n g_i^{a_i}$ to denote the multi-exponentiation.

We use λ to denote the security parameter and will drop it from the notation when it is implicit. Let PPT denote probabilistic polynomial time. We use $\text{negl}(\lambda)$ to denote a negligible function in λ .

We use $y \xleftarrow{\$} S$ to denote the assignment of a uniformly random element in set S to y . We use $y := c$ to denote the assignment of the value c to y . When A is an algorithm, we use $y \leftarrow A(x)$ to denote the assignment of the output of A given input x to y . Let $|S|$ denote the size of set S .

2.2 Quadratic voting

Quadratic voting [36] is a rated voting method where voters express the degree of their preferences by casting v votes at a cost of v^2 . We consider quadratic voting with a fixed budget and multiple choices, where each voter is given a fixed budget to allocate across multiple choices by distributing their votes accordingly.

Definition 2.1 (Fixed-budget multiple-choice quadratic voting [24]).

A fixed-budget multiple-choice QV has the following components:

- Δ : the set of voters.
- Ω : the finite set of choices.
- $v \in \mathbb{Z}^{|\Delta| \times |\Omega|}$: the strategy profile, where v_{ω}^{δ} is the number of votes voter δ casts for choice ω . The positive number represents support while the negative number represents opposition.

- $B \in \mathbb{N}$: a budget for each voter to spend in the election. In DAOs, the budget corresponds to the number of governance tokens each voter holds, which may differ across voters.
- $t_{\delta} = \sum_{\omega \in \Omega} (v_{\omega}^{\delta})^2$: the total price that voter $\delta \in \Delta$ needs to pay. It should hold that $t_{\delta} \leq B$.
- $s_{\omega} = \sum_{\delta \in \Delta} v_{\omega}^{\delta}$: the total number of votes for choice $\omega \in \Omega$.
- W : the set of choices that have the highest s_{ω} . The winning choice is selected uniformly at random from W , implying that the probability choice $\omega \in \Omega$ is elected is $\mathbb{P}_{\omega}(v) = \begin{cases} 1/|W|, & \text{if } \omega \in W, \\ 0, & \text{otherwise.} \end{cases}$

We aim to design a decentralized self-tallying quadratic voting with maximal ballot secrecy, where each voter's ballot $(v_{\omega}^{\delta})_{\omega \in \Omega}$ remains hidden from the public, yet everyone could obtain the tallying result $(s_{\omega})_{\omega \in \Omega}$ in a verifiable manner.

2.3 Interactive zero-knowledge argument of knowledge

Let $\mathcal{R}$ be a polynomial-time-decidable ternary relation. For a public parameter pp , we call w a witness for an instance x if $(\text{pp}, x; w) \in \mathcal{R}$. An interactive zero-knowledge argument of knowledge (ZKAoK) for $\mathcal{R}$ allows a prover to convince a verifier of knowing w such that $(\text{pp}, x; w) \in \mathcal{R}$. It satisfies 1) Perfect completeness: an honest prover with a valid witness always convinces an honest verifier; 2) Computational witness-extended emulation/knowledge soundness: if a prover convinces a verifier, there exists an extractor to extract a valid witness; 3) Perfect (special) honest-verifier zero-knowledge: there exists a simulator to produce transcripts indistinguishable from real proofs without a witness. Their formal definitions are deferred to Appendix A.

Interactive public-coin zero-knowledge arguments of knowledge can be made non-interactive using the Fiat-Shamir transformation [22], by replacing the verifier's challenges with hashes of the transcript (including the statement) up to that point. In this paper, we also require the hash input to include a voter's unique identity [29] to prevent replay attacks.

2.4 Commitment schemes

Definition 2.2 (Commitment scheme). A commitment scheme C is a tuple $C = (\text{Setup}, \text{Com}, \text{Open})$ of PPT algorithms where

- $ck \leftarrow \text{Setup}(1^{\lambda})$: on input a security parameter λ , outputs a commitment key ck specifying a message space $\mathcal{M}$.
- $(C, r) \leftarrow \text{Com}(ck, m)$: on input a commitment key ck , a message $m \in \mathcal{M}$, outputs a commitment C , (optionally) a secret opening hint r . Sometimes we omit ck for simplicity when writing Com .
- $1/0 \leftarrow \text{Open}(ck, C, m, r)$: on input ck, C, m, r , outputs 1/0 to denote that C is indeed a commitment to m or not.

A commitment scheme satisfies binding and hiding. Their formal definitions are deferred to Appendix B.

Definition 2.3 (Polynomial commitment scheme). A polynomial commitment scheme $\mathcal{PC}$ is a tuple $\mathcal{PC} = (\text{Setup}, \text{PCom}, \text{Open}, \text{Eval})$ where $(\text{Setup}, \text{PCom}, \text{Open})$ is a commitment scheme for the message space $\mathbb{F}[\ell]$ of ℓ -variate multilinear polynomials over $\mathbb{F}$, and

- $1/0 \leftarrow \text{Eval} = (\text{Setup}, \mathcal{P}, \mathcal{V})$ is an interactive ZKAoK between a prover $\mathcal{P}$ and a verifier $\mathcal{V}$ for relation $\{(ck, \ell, C, \mathbf{u}, v; \tilde{f}(X), r) : \text{Open}(ck, C, \tilde{f}, r) = 1 \wedge \tilde{f} \in \mathbb{F}[\ell] \wedge \tilde{f}(\mathbf{u}) = v\}$.

LEMMA 2.4 (SCHWARTZ-ZIPPEL LEMMA [51]). Let $\tilde{f} : \mathbb{F}^\ell \rightarrow \mathbb{F}$ be a non-zero ℓ -variate polynomial of total degree d . Let S be any finite subset of $\mathbb{F}$. Then for $\mathbf{r} \xleftarrow{\$} S^\ell$, $\Pr[\tilde{f}(\mathbf{r}) = 0] \leq \frac{d}{|S|}$.

LEMMA 2.5 (MULTILINEAR EXTENSION [12]). For every function $f : \{0, 1\}^\ell \rightarrow \mathbb{F}$, there is a unique ℓ -variate multilinear polynomial $\tilde{f} : \mathbb{F}^\ell \rightarrow \mathbb{F}$ such that $\tilde{f}(\mathbf{b}) = f(\mathbf{b})$ for all $\mathbf{b} \in \{0, 1\}^\ell$. We call $\tilde{f}$ the multilinear extension (MLE) of function f , and $\tilde{f}$ can be expressed as $\tilde{f}(X) = \sum_{\mathbf{b} \in \{0, 1\}^\ell} f(\mathbf{b}) \cdot \tilde{eq}(X, \mathbf{b})$, where $\tilde{eq}(X, \mathbf{b}) = \prod_{i=1}^\ell (b_i X_i + (1 - b_i)(1 - X_i))$.

3 Decentralized quadratic voting

We assume the presence of an authenticated broadcast channel for all voters [27, 29, 34], which can be realized through the use of a public bulletin board like blockchain and digital signatures generated from wallets as implemented in real-world DAO elections [14]. Let $\mathbb{G}$ denote a finite cyclic group of prime order p where the decision Diffie-Hellman (DDH) assumption holds. This can be realized by using standard elliptic curves (e.g., Secp256k1). Let $\mathbb{F}_p$ denote a finite field of order p . Suppose n_v voters and n_c choices. The i -th voter is denoted as V_i and has t_i tokens. The j -th choice is denoted as C_j . Let $g, h, (g_1, h_1), (g_2, h_2), \dots, (g_{n_c}, h_{n_c})$ be several uniformly random elements in $\mathbb{G}$.

3.1 Security requirements

We consider the following essential security requirements as in previous works [27, 29, 34, 38, 40, 42, 61]³.

Maximal ballot secrecy. The partial tally of voters' ballots is only accessible to the coalition of remaining voters. For n_v voters, consider an adversary $\mathcal{A}$ capable of corrupting at most $n_v - 2$ voters. $\mathcal{A}$ sends two different plain votes to a challenger C , who then generates two ballots for two uncorrupted voters. If $\mathcal{A}$ cannot distinguish which ballot corresponds to which plain vote, maximal ballot secrecy holds.

As in [38], we formally define the security model for maximal ballot secrecy by an interaction game between an adversary $\mathcal{A}$ and a challenger C . For n_v voters, we assume $\mathcal{A}$ could corrupt at most $n_v - 2$ voters, since the compromise of $n_v - 1$ voters will directly lead to the revealing of the last voter based on the tallying result. $\mathcal{A}$ could adaptively query the corrupted voters' ballots. Then, $\mathcal{A}$ sends two different plain votes $(v^* := (v_1^*, \dots, v_{n_c}^*), \hat{v}^* := (\hat{v}_1^*, \dots, \hat{v}_{n_c}^*))$ to C as a challenge. As a response, C generates two ballots for the two uncorrupted voters (V_s, V_t) . $\mathcal{A}$ should distinguish which of

the two ballots is from the vote $\hat{v}^*$. The detailed security model between $\mathcal{A}$ and C is as follows.

Initial. $\mathcal{A}$ declares two target voters (V_s, V_t) to be challenged upon. The other voters are regarded as corrupted by $\mathcal{A}$.

Setup. C generates the secret and public key pairs for each voter, and then forwards all the public keys and the corrupted voters' secret keys to $\mathcal{A}$.

Query. $\mathcal{A}$ adaptively chooses the votes and queries the corresponding ballots generated by any voters other than (V_s, V_t) .

Challenge. $\mathcal{A}$ sends two different plain votes $(v^* := (v_1^*, \dots, v_{n_c}^*), \hat{v}^* := (\hat{v}_1^*, \dots, \hat{v}_{n_c}^*))$ to C as a challenge. C randomly selects $b \in \{s, t\}$ and assigns the vote $\hat{v}^*$ to uncorrupted voter V_b . C assigns v^* to another uncorrupted voter.

Tally. $\mathcal{A}$ computes the tally based on the collected ballots.

Guess. $\mathcal{A}$ outputs a guess $b' \in \{s, t\}$ to determine which one between V_s and V_t has cast the ballot of $\hat{v}^*$.

Definition 3.1 (Maximal ballot secrecy). The voting scheme achieves maximal ballot secrecy if for any probabilistic polynomial time adversary, $|\Pr[b' = b] - 1/2| \leq \epsilon$, where ϵ is negligible.

Self-tallying. After all the voters cast their ballots, anyone can compute the election result.

Dispute-freeness. Anyone can verify whether voters follow the protocol, ensuring public verifiability of the result.

3.2 Scheme description

Our voting scheme is inspired by [29, 40] but previous works only support simple yes/no voting. Quadratic voting is significantly more complex: in particular, proving the well-formedness of a QV ballot. QV-net has a voting phase and a tallying phase, as detailed in Figure 1.

3.2.1 Voting phase. The voting phase has two rounds. In the first round, each voter V_i samples a secret key $x_{i,j}$ and computes a public key $y_{i,j} := h_j^{x_{i,j}}$ for choice C_j , where $i \in [n_v], j \in [n_c]$. To prevent a voter from learning other voters' ballots and ensure the correct form of public keys, we require each V_i to generate a proof $\pi_i^{(dl)}$ proving knowledge of secret keys $\{x_{i,j}\}_{j \in [n_c]}$. The corresponding relation is defined as $\mathcal{R}_{dl} := \{(n \in \mathbb{N}, \mathbf{h} \in \mathbb{G}^n, \mathbf{y} \in \mathbb{G}^n; \mathbf{x} \in \mathbb{F}_p^n) : y_j = h_j^{x_j} \text{ for } j \in [n]\}$. We use an AND composition of Schnorr protocol [50] to generate the proof $\pi_i^{(dl)}$ for $\mathcal{R}_{dl}$ (see Appendix C). Finally, each V_i publishes $\{y_{i,j}\}_{j \in [n_c]}, \pi_i^{(dl)}$ to the bulletin board.

In the second round, each V_i collects $\{y_{k,j}\}_{k \in [n_v] \setminus \{i\}, j \in [n_c]}$ from the bulletin board and computes $Y_{i,j} := \frac{\prod_{k=1}^{i-1} y_{k,j}}{\prod_{k=i+1}^{n_v} y_{k,j}}$ for $j \in [n_c]$. V_i then commits to their vote $v_{i,j}$ for choice C_j as $B_{i,j} := g_j^{v_{i,j}} Y_{i,j}^{x_{i,j}}$. Note that since $(g_1, h_1), \dots, (g_{n_c}, h_{n_c})$ are uniformly random, $(g_1, Y_{i,1}), \dots, (g_{n_c}, Y_{i,n_c})$ remain uniformly random even if all voters collude. To ensure correct tallying, we require V_i to prove that $x_{i,j}$ underlying $B_{i,j}$ is the secret key corresponding to the public key $y_{i,j}$. The corresponding relation is defined as $\mathcal{R}_{eq} := \{(n \in \mathbb{N}, \mathbf{h}, \mathbf{g}, \mathbf{Y} \in \mathbb{G}^n, \mathbf{y}, \mathbf{B} \in \mathbb{G}^n; \mathbf{v}, \mathbf{x} \in \mathbb{F}_p^n) : y_j = h_j^{x_j} \wedge B_j = g_j^{v_j} Y_j^{x_j} \text{ for } j \in [n]\}$. We use an AND composition of the Σ protocol for discrete logarithm equality to generate a proof $\pi_i^{(eq)}$ for relation $\mathcal{R}_{eq}$ (see Appendix D).

³Some works [27, 33, 34, 38, 40, 61] also consider fairness, meaning that nobody has the priority to get a partial tally ahead of schedule. They provide standard methods for achieving this, which are also applicable to QV-net.

Voting phase: For $i \in [n_v]$, each voter V_i proceeds as follows.

Round 1. For $j \in [n_c]$, V_i samples $x_{i,j} \xleftarrow{\$} \mathbb{F}_p$ and computes $y_{i,j} := h_j^{x_{i,j}}$. Then V_i invokes a protocol for $\mathcal{R}_{dl}$ on input $(n_c, (h_1, \dots, h_{n_c}), (y_{i,1}, \dots, y_{i,n_c}); (x_{i,1}, \dots, x_{i,n_c}))$ to generate a proof $\pi_i^{(dl)}$ proving knowledge of exponents $\{x_{i,j}\}_{j \in [n_c]}$ (see Appendix C for details). V_i publishes $\{y_{i,j}\}_{j \in [n_c]}, \pi_i^{(dl)}$ to the bulletin board.

Round 2. For $j \in [n_c]$, V_i computes $Y_{i,j} := \frac{\prod_{k=1}^{j-1} y_{k,j}}{\prod_{k=i+1}^{n_v} y_{k,j}}$, and then computes $B_{i,j} := g_j^{v_{i,j}} Y_{i,j}^{x_{i,j}}, B_i := g^{u_i} h^{r_i}$, where $v_{i,j}$ is the plain vote for choice C_j and $u_i = \sum_{j \in [n_c]} v_{i,j}^2, r_i \xleftarrow{\$} \mathbb{F}_p$. V_i then invokes the following ZKAoKs to generate proofs.

1. V_i invokes a ZKAoK for $\mathcal{R}_{eq}$ on input $(n_c, (h_1, \dots, h_{n_c}), (g_1, \dots, g_{n_c}), (Y_{i,1}, \dots, Y_{i,n_c}), (y_{i,1}, \dots, y_{i,n_c}), (B_{i,1}, \dots, B_{i,n_c}); (v_{i,1}, \dots, v_{i,n_c}), (x_{i,1}, \dots, x_{i,n_c}))$ to generate a proof $\pi_i^{(eq)}$ proving the consistency of $\{x_{i,j}\}_{j \in [n_c]}$ underlying $\{y_{i,j}\}_{j \in [n_c]}$ and $\{B_{i,j}\}_{j \in [n_c]}$ (see Appendix D for details).
2. V_i invokes a ZKAoK for $\mathcal{R}_{ss}$ on input $(n_c, (g_1, \dots, g_{n_c}), (Y_{i,1}, \dots, Y_{i,n_c}), g, h, \prod_{j=1}^{n_c} B_{i,j}, B_i; (v_{i,1}, \dots, v_{i,n_c}), (x_{i,1}, \dots, x_{i,n_c}), u_i, r_i)$ to generate a proof $\pi_i^{(ss)}$ proving the sum-of-square relation $u_i = \sum_{j \in [n_c]} v_{i,j}^2$ (see Section 4 for details).
3. V_i invokes a ZKAoK for $\mathcal{R}_{ar}$ on input $(n_c, \lfloor \sqrt{t_i} \rfloor, t_i, (g_1, \dots, g_{n_c}), g, (Y_{i,1}, \dots, Y_{i,n_c}), h, (B_{i,1}, \dots, B_{i,n_c}), B_i; (v_{i,1}, \dots, v_{i,n_c}), u_i, (x_{i,1}, \dots, x_{i,n_c}), r_i)$ to generate a proof $\pi_i^{(ar)}$ proving range relation $v_{i,j} \in [-\lfloor \sqrt{t_i} \rfloor, \lfloor \sqrt{t_i} \rfloor], u_i \in [0, t_i]$ for $j \in [n_c]$ (see Section 5 for details).

Finally, V_i publishes $\{B_{i,j}\}_{j \in [n_c]}, B_i, \pi_i^{(eq)}, \pi_i^{(ss)}, \pi_i^{(ar)}$ to the bulletin board.

Tallying phase: Every voter and any third party can check the validity of all the proofs to ensure that no badly formed vote has been cast to distort the tally. Then anyone can compute $\prod_{i \in [n_v]} B_{i,j} = g_j^{\sum_{i \in [n_v]} v_{i,j}}$ for $j \in [n_c]$. By exhaustive search or Shanks' baby-step giant-step algorithm [53], one can obtain the total vote $\sum_{i \in [n_v]} v_{i,j}$ for choice C_j .

Figure 1: The QV-net scheme

Since V_i only has t_i tokens, as per Definition 2.1, we require V_i to prove that each vote $v_{i,j}$ is within the range $[-\lfloor \sqrt{t_i} \rfloor, \lfloor \sqrt{t_i} \rfloor]$ and $\sum_{j \in [n_c]} v_{i,j}^2 \leq t_i$. We separate this statement into two statements. We let V_i additionally compute $u_i := \sum_{j \in [n_c]} v_{i,j}^2$ and commits to it as $B_i := g^{u_i} h^{r_i}$ where r_i is a random field element. Then V_i needs to prove the sum-of-square relation $u_i = \sum_{j \in [n_c]} v_{i,j}^2$ and the range relation $v_{i,j} \in [-\lfloor \sqrt{t_i} \rfloor, \lfloor \sqrt{t_i} \rfloor], u_i \in [0, t_i]$ for $j \in [n_c]$.

Since relation $\mathcal{R}_{eq}$ has ensured the correct form of $\{B_{i,j}\}_{j \in [n_c]}$, the equality $\prod_{j \in [n_c]} B_{i,j} = \prod_{j \in [n_c]} g_j^{v_{i,j}} Y_{i,j}^{x_{i,j}}$ implies $B_{i,j} = g_j^{v_{i,j}} Y_{i,j}^{x_{i,j}}$ for $j \in [n_c]$ under the discrete logarithm assumption. Thus, for ease of proving, we define the sum-of-square relation as $\mathcal{R}_{ss} := \{(n \in \mathbb{N}, g, Y \in \mathbb{G}^n, g, h \in \mathbb{G}, B, \hat{B} \in \mathbb{G}; v, x \in \mathbb{F}_p^n, u, r \in \mathbb{F}_p) : B = g^v Y^x \wedge \hat{B} = g^u h^r \wedge u = \sum_{j \in [n]} v_j^2\}$. We design a succinct ZKAoK for relation $\mathcal{R}_{ss}$ in Section 4 to generate the corresponding proof $\pi_i^{(ss)}$, where B is assigned the value $\prod_{j \in [n_c]} B_{i,j}$.

We formally define the range relation as $\mathcal{R}_{ar} := \{(n, s, t \in \mathbb{N}, g, h \in \mathbb{G}^{n+1}, B \in \mathbb{G}^{n+1}; v, x \in \mathbb{F}_p^{n+1}) : \text{for } j \in [n+1], B_j = g_j^{v_j} h_j^{x_j} \wedge v_1, \dots, v_n \in [-s, s] \wedge v_{n+1} \in [0, t]\}$. We design a protocol for relation $\mathcal{R}_{ar}$ in Section 5 to generate the corresponding proof $\pi_i^{(ar)}$.

Finally, in the second round, each voter V_i publishes $\{B_{i,j}\}_{j \in [n_c]}, B_i, \pi_i^{(eq)}, \pi_i^{(ss)}, \pi_i^{(ar)}$ to the bulletin board.

3.2.2 Tallying phase. Anyone can read the data on the bulletin board to verify proofs and perform tallying. Specifically, it can be verified that $\prod_{i \in [n_v]} Y_{i,j}^{x_{i,j}} = \prod_{i \in [n_v]} \left(h_j^{\sum_{k=1}^{j-1} x_{k,j} - \sum_{k=i+1}^{n_v} x_{k,j}} \right)^{x_{i,j}} = h_j^{\sum_{i \in [n_v]} x_{i,j} \cdot (\sum_{k=1}^{j-1} x_{k,j} - \sum_{k=i+1}^{n_v} x_{k,j})} = h_j^0 = 1_{\mathbb{G}}$ for $j \in [n_c]$. Thus, one can compute $\prod_{i \in [n_v]} B_{i,j} = \prod_{i \in [n_v]} g_j^{v_{i,j}} Y_{i,j}^{x_{i,j}} = g_j^{\sum_{i \in [n_v]} v_{i,j}}$. Since

the tally $\sum_{i \in [n_v]} v_{i,j}$ for C_j is relatively small, it can be computed by exhaustive search or Shanks' baby-step giant-step algorithm [53].

3.3 Security analysis

THEOREM 3.2. *If invoked protocols for relations $\mathcal{R}_{dl}, \mathcal{R}_{eq}, \mathcal{R}_{ss}, \mathcal{R}_{ar}$ are ZKAoKs, then QV-net achieves maximal ballot secrecy under the DDH assumption and satisfies self-tallying as well as dispute-freeness.*

PROOF. We present the proof below.

Maximal ballot secrecy. We define a sequence of games between an adversary $\mathcal{A}$ and a challenger C to prove this property. We denote $\Pr[\text{Win}_i]$ as the winning probability of $\mathcal{A}$ (output a correct guess) in Game _{i} .

Game₀: This is the original game defined in Section 3.1. $\mathcal{A}$ chooses two target voters V_s, V_t to challenge upon and forwards them to C . Suppose that $1 \leq s < t \leq n_v$. Let $(\{B_{s,j}\}_{j \in [n_c]}, B_s, \pi_s), (\{B_{t,j}\}_{j \in [n_c]}, B_t, \pi_t)$ denote the challenge ballots generated by C , where π_k represents all the corresponding proofs for $k \in \{s, t\}$. Due to the perfect hiding of the Pedersen commitment, B_s, B_t are uniformly random for $\mathcal{A}$. From the definition of this game, we have $\Pr[\text{Win}_0] = \Pr[b' = b]$.

Game₁: This game is the same as Game₀ with one difference. Specifically, C runs the simulators of the invoked zero-knowledge arguments of knowledge to replace proofs π_s, π_t with the simulated proofs π'_s, π'_t without using the real witness. Due to the zero-knowledge of the ZKAoK, Game₀ and Game₁ are indistinguishable from $\mathcal{A}$'s view. Thus, we have $|\Pr[\text{Win}_1] - \Pr[\text{Win}_0]| \leq \epsilon_{zk}$, where ϵ_{zk} is negligible.

Game₂: This game is the same as Game₁ with one difference, where C replaces $\{B_{s,j}\}_{j \in [n_c]}, \{B_{t,j}\}_{j \in [n_c]}$ with random elements $\{B'_{s,j}\}_{j \in [n_c]}, \{B'_{t,j}\}_{j \in [n_c]}$ without changing the tallying result. In

more detail, suppose that the public keys for V_s, V_t are $\{y_{s,j} = h_j^{x_{s,j}}\}_{j \in [n_c]}, \{y_{t,j} = h_j^{x_{t,j}}\}_{j \in [n_c]}$. We have $B_{s,j} = g_j^{v_{s,j}} Y_{s,j}^{x_{s,j}} = g_j^{v_{s,j}} \cdot S_j / h_j^{x_{s,j} \cdot x_{t,j}}, B_{t,j} = g_j^{v_{t,j}} Y_{t,j}^{x_{t,j}} = g_j^{v_{t,j}} \cdot T_j \cdot h_j^{x_{s,j} \cdot x_{t,j}}$, where $S_j = \sum_{i=1}^{s-1} x_{i,j} - \sum_{i=s+1}^{t-1} x_{i,j} - \sum_{i=t+1}^{n_v} x_{i,j}$, $T_j = \sum_{i=1}^{s-1} x_{i,j} + \sum_{i=s+1}^{t-1} x_{i,j} - \sum_{i=t+1}^{n_v} x_{i,j}$. Then C sets $B'_{s,j} := g_j^{v_{s,j}} \cdot S_j / R_j, B'_{t,j} := g_j^{v_{t,j}} \cdot T_j \cdot R_j$ where $R_j = h_j^{r_j}$

with $r_j \xleftarrow{\$} \mathbb{F}_p$. We have $B_{s,j} \cdot B_{t,j} = B'_{s,j} \cdot B'_{t,j}$, which implies that tallying is not affected. Due to the DDH assumption, Game_1 and Game_2 are indistinguishable from $\mathcal{A}$'s view. Thus, we have $|\Pr[\text{Win}_2] - \Pr[\text{Win}_1]| \leq \epsilon_{DDH}$, where ϵ_{DDH} is negligible.

Since the challenge ballots generated by C in Game_2 are all uniformly random for $\mathcal{A}$, we have $\Pr[\text{Win}_2] = 1/2$. Therefore, $|\Pr[b' = b] - 1/2| \leq \epsilon_{zk} + \epsilon_{DDH}$. We conclude that QV-net achieves maximal ballot secrecy.

Self-tallying. The invoked ZKAoK for relations $\mathcal{R}_{dl}, \mathcal{R}_{eq}$ ensures that $\prod_{i \in [n_v]} Y_{i,j}^{x_{i,j}} = 1_{\mathbb{G}}$ for $j \in [n_c]$, which indicates that $\prod_{i \in [n_v]} C_{i,j} = g_j^{\sum_{i \in [n_v]} v_{i,j}}$. Then with the data on the bulletin board, anyone can compute the tallying result by exhaustive search or Shanks' baby-step giant-step algorithm.

Dispute-freeness. Since invoked ZKAoKs are publicly verifiable, anyone can verify the proofs on the bulletin board to ensure that voters follow the voting scheme faithfully. Since we use a public authenticated channel, any attempt to cast more than one vote can be detected by others. Hence, the dispute-freeness holds. $\square$

4 A succinct ZKAoK for sum-of-square relation

In this section, we propose a succinct ZKAoK for relation $\mathcal{R}_{ss} := \{(n \in \mathbb{N}, g, Y \in \mathbb{G}^n, g, h \in \mathbb{G}, B, \hat{B} \in \mathbb{G}; v, x \in \mathbb{F}_p^n, u, r \in \mathbb{F}_p) : B = g^{vY} \wedge \hat{B} = g^{uY} \wedge u = \sum_{j \in [n]} v_j^2\}$.

4.1 Overview

We treat the sum-of-square relation as an inner product relation and utilize the sum-check protocol [39] to complete the proof. Specifically, we rewrite $u = \sum_{j \in [n]} v_j^2$ as $u = \langle v, v \rangle$ and interpret the vector v as a function $f : \{0, 1\}^\ell \rightarrow \mathbb{F}_p$, where an input in $\{0, 1\}^\ell$ represents the binary encoding of an index $j \in \{0, \dots, n-1\}$, and the output is v_{j+1}^4 . By Lemma 2.5, we construct a multilinear polynomial $\tilde{f}(X) := \langle v, (\tilde{eq}(X, y))_{y \in \{0,1\}^\ell} \rangle$ to encode v such that $\tilde{f}(b) = f(b)$ for all $b \in \{0, 1\}^\ell$. Then to prove $u = \sum_{j \in [n]} v_j^2$, it is equivalent to proving $u = \sum_{b \in \{0,1\}^\ell} \tilde{f}(b) \cdot \tilde{f}(b)$, which is exactly a sum-check relation and can be proved by the sum-check protocol.

We briefly review the sum-check protocol according to [35]. It is an interactive protocol allowing a prover $\mathcal{P}$ to convince a verifier $\mathcal{V}$ that $T = \sum_{b \in \{0,1\}^\ell} \tilde{g}(b)$, where $\tilde{g} : \mathbb{F}^\ell \rightarrow \mathbb{F}$ is an ℓ -variate polynomial, and the degree of each variable X_i is at most d_i for $i \in [\ell]$. In the protocol, $\mathcal{V}$ takes as input randomness $r \in \mathbb{F}^\ell$ and interacts with $\mathcal{P}$ over ℓ rounds. At the end of the interaction, $\mathcal{V}$ outputs a claim about the evaluation $\tilde{g}(r)$. Let $\langle \mathcal{P}, \mathcal{V}(r) \rangle$ denote the interaction between $\mathcal{P}$ and $\mathcal{V}$ with $\mathcal{V}$'s randomness r . We treat $\langle \mathcal{P}, \mathcal{V}(r) \rangle$ as a function that on input $(\tilde{g}, \ell, d, T)$ outputs the claimed evaluation to be checked, i.e., $c \leftarrow \langle \mathcal{P}, \mathcal{V}(r) \rangle(\tilde{g}, \ell, d, T)$. By completeness, if $T = \sum_{b \in \{0,1\}^\ell} \tilde{g}(b)$, then $\tilde{g}(r) = c$. By soundness,

⁴We assume that $n = 2^\ell$; otherwise, we can pad vectors to satisfy it.

if $\tilde{g}(r) = c$, then $T = \sum_{b \in \{0,1\}^\ell} \tilde{g}(b)$ with overwhelming probability. The formal security definitions of sum-check and full specification of $c \leftarrow \langle \mathcal{P}, \mathcal{V}(r) \rangle(\tilde{g}, \ell, d, T)$ are deferred to Appendix E.

However, applying the sum-check protocol to prove the relation $u = \sum_{b \in \{0,1\}^\ell} \tilde{f}(b) \cdot \tilde{f}(b)$ poses several challenges. Next, we outline these challenges and propose solutions.

4.1.1 Verifying the output claim. After executing $c \leftarrow \langle \mathcal{P}, \mathcal{V}(r) \rangle(\tilde{f}, \ell, (2, 2, \dots, 2), u)$ to prove $u = \sum_{b \in \{0,1\}^\ell} \tilde{f}(b) \cdot \tilde{f}(b)$, the verifier must verify $\tilde{f}(r) \cdot \tilde{f}(r) = c$. However, since polynomial $\tilde{f}$ encodes a private vector v , the verifier cannot compute $\tilde{f}(r)$ directly. To address this, we let the prover reveal $\tilde{f}(r)$ and prove its correctness using a multilinear polynomial commitment scheme (PCS). In this paper, we consider the discrete-logarithm-based PCS for its transparency and small proof size. Commitments to vectors are interpreted as polynomial commitments. For a multilinear polynomial $\tilde{a}(X) = \langle a, (\tilde{eq}(X, y))_{y \in \{0,1\}^\ell} \rangle$, we define $C_a \leftarrow \text{PCom}(\tilde{a}) = g^a Y^{r_a}$, where $g, Y \in \mathbb{G}^n, r_a \in \mathbb{F}_p^n$ are uniformly random. This PCS is clearly hiding. To open $\tilde{a}$ at point $e = (e_1, \dots, e_\ell)$ that evaluates v_a (i.e., perform the Eval protocol), the prover proves that $C_a = g^a Y^{r_a} \wedge v_a = \langle a, (\tilde{eq}(e, y))_{y \in \{0,1\}^\ell} \rangle + \langle r_a, \mathbf{0}_n \rangle$ by using variants of Bulletproofs [1, 5, 56] (see Appendix F for details), where $\mathbf{0}_n$ is a zero vector of length n . Under this PCS, the element B in relation $\mathcal{R}_{ss}$ is exactly a polynomial commitment to polynomial $\tilde{f}$. Thus, the PCS enables the verifier to obtain $\tilde{f}(r)$ in a verifiable manner. However, since $\tilde{f}$ depends on the witness v , revealing $\tilde{f}(r)$ could violate zero-knowledge. Moreover, the sum-check and Eval protocols may also leak information about the witness.

4.1.2 Achieving zero-knowledge. We follow guidelines in [8, 55, 60] to achieve zero-knowledge, which requires masking $\tilde{f}$ with an appropriate random polynomial, using a zero-knowledge sum-check, and using a hiding PCS with a zero-knowledge Eval protocol. **Masking the witness polynomial.** Since the witness polynomial $\tilde{f}$ will be opened only once, according to [8], it suffices to sample a random univariate polynomial $\tilde{R}(X_1) := w$ and mask $\tilde{f}$ as $\tilde{f}^*(X) := \tilde{f}(X) + \tilde{R}(X_1) \cdot \tilde{Z}(X_1)$, where $w \xleftarrow{\$} \mathbb{F}_p, \tilde{Z}(X_1) = X_1 \cdot (1 - X_1)$. We have $\tilde{f}^*(b) = \tilde{f}(b) = f(b)$ for all $b \in \{0, 1\}^\ell$. Thus, to prove $u = \sum_{b \in \{0,1\}^\ell} \tilde{f}(b) \cdot \tilde{f}(b)$, it is equivalent to proving $u = \sum_{b \in \{0,1\}^\ell} \tilde{f}^*(b) \cdot \tilde{f}^*(b)$. At the end of the sum-check protocol, the prover reveals $\tilde{f}^*(r)$ to the verifier⁵. Since $\tilde{f}^*$ involves the random polynomial $\tilde{R}$, the evaluation $\tilde{f}^*(r)$ is uniformly random, revealing no information about $\tilde{f}$. However, unlike [8], the verifier must ensure that $\tilde{R} \cdot \tilde{Z}$ is well-formed; otherwise, a malicious prover could convince the verifier of a false statement. For example, if $u \neq \sum_{j \in [n]} v_j^2$, then $u \neq \sum_{b \in \{0,1\}^\ell} \tilde{f}(b) \cdot \tilde{f}(b)$. A malicious prover could carefully craft $\tilde{R} \cdot \tilde{Z}$ such that $u = \sum_{b \in \{0,1\}^\ell} \tilde{f}^*(b) \cdot \tilde{f}^*(b)$, and due to the completeness of the sum-check protocol, the verifier always accepts. This attack occurs since relation $\mathcal{R}_{ss}$ requires the prover to know a private vector v such that $u = \sum_{j \in [n]} v_j^2$ and bind it to a public commitment B . In contrast, Chen et al. [8] avoid this issue by requiring only knowledge of a witness without binding it to any public commitment.

⁵For zero-knowledge, the first item r_1 of r is sampled from $\mathbb{F}_p \setminus \{0, 1\}$ rather than $\mathbb{F}_p$.

To prevent the above attack, we modify the polynomial $\tilde{R}$ to be a multilinear polynomial and let the verifier construct a new polynomial $\tilde{Z}(X) := \prod_{i=1}^{\ell} X_i(1 - X_i)$ by himself⁶. Specifically, the prover samples $\tilde{R}(X) := \langle (w, \mathbf{0}_{n-1}), (\tilde{e}q(X, \mathbf{y}))_{\mathbf{y} \in \{0,1\}^{\ell}} \rangle$ and commits to it as $C_R \leftarrow \text{PCom}(\tilde{R}) = \mathbf{g}^{(w, \mathbf{0}_{n-1})} Y^{\mathbf{r}_w}$, where $w \xleftarrow{\$} \mathbb{F}_p, \mathbf{r}_w \xleftarrow{\$} \mathbb{F}_p^n$. The prover then masks $\tilde{f}$ as $\tilde{f}^*(X) := \tilde{f}(X) + \tilde{R}(X) \cdot \tilde{Z}(X)$. We have $\tilde{f}^*(\mathbf{b}) = \tilde{f}(\mathbf{b}) = f(\mathbf{b})$ for all $\mathbf{b} \in \{0, 1\}^{\ell}$. As in [55], we require that $\mathbf{r}$ is sampled from $\mathbb{F}_p^{\ell} \setminus \{0, 1\}^{\ell}$. Thus, $\tilde{f}^*(\mathbf{r})$ remains uniformly random and reveals nothing about $\tilde{f}$. Based on the homomorphic property of the PCS, the verifier could compute the commitment to $\tilde{f}^*$ as $C_{f^*} := B \cdot C_R^{\tilde{Z}(\mathbf{r})}$. With C_{f^*} , the verifier participates in the Eval protocol of the PCS to verify the evaluation $\tilde{f}^*(\mathbf{r})$, which guarantees that $\tilde{f}^*$ is well-formed and that $\tilde{f}$ is bound to B . Then, due to the soundness of the sum-check protocol, after checking $\tilde{f}^*(\mathbf{r}) \cdot \tilde{f}^*(\mathbf{r}) = c$, the verifier could ensure that $u = \sum_{\mathbf{b} \in \{0,1\}^{\ell}} \tilde{f}^*(\mathbf{b}) \cdot \tilde{f}^*(\mathbf{b})$ and thereby $u = \sum_{\mathbf{b} \in \{0,1\}^{\ell}} \tilde{f}(\mathbf{b}) \cdot \tilde{f}(\mathbf{b})$ with overwhelming probability.

Using a zero-knowledge sum-check. After masking the witness polynomial, the prover and verifier execute the sum-check protocol $c \leftarrow \langle \mathcal{P}, \mathcal{V}(\mathbf{r}) \rangle (\tilde{f}^* \cdot \tilde{f}^*, \ell, (6, 6, \dots, 6), u)$ to prove $u = \sum_{\mathbf{b} \in \{0,1\}^{\ell}} \tilde{f}^*(\mathbf{b}) \cdot \tilde{f}^*(\mathbf{b})$. During this process, the prover sends univariate polynomials derived from $\tilde{f}^* \cdot \tilde{f}^*$ in each round, which may leak information about the witness. To prevent this, we take the approach in [60] to make the sum-check protocol zero-knowledge.

The prover selects a random polynomial $\tilde{g}^*(X) := v_0^* + \tilde{g}_1^*(X_1) + \tilde{g}_2^*(X_2) + \dots + \tilde{g}_{\ell}^*(X_{\ell})$, where $\tilde{g}_i^*(X_i) = \sum_{j=1}^6 v_{i,j}^* X_i^j$ for $i \in [\ell]$ and all coefficients are uniformly random. The prover then computes and sends $u^* := \sum_{\mathbf{b} \in \{0,1\}^{\ell}} \tilde{g}^*(\mathbf{b})$, $C_{g^*} \leftarrow \text{PCom}(\tilde{g}^*)$ to the verifier.

The verifier samples and sends $\rho \xleftarrow{\$} \mathbb{F}_p^*$ to the prover, and then computes $u + \rho u^*$. Next, the prover and verifier run the plain sum-check protocol $c \leftarrow \langle \mathcal{P}, \mathcal{V}(\mathbf{r}) \rangle (\tilde{f}^* \cdot \tilde{f}^* + \rho \cdot \tilde{g}^*, \ell, (6, 6, \dots, 6), u + \rho u^*)$ without zero-knowledge. Finally, the verifier engages in the Eval protocol of PCS to obtain $v_{f^*} := \tilde{f}^*(\mathbf{r})$, $v_{g^*} := \tilde{g}^*(\mathbf{r})$ and checks $c = v_{f^*} \cdot v_{f^*} + \rho \cdot v_{g^*}$. Due to the random polynomial $\tilde{g}^*$, the plain sum-check protocol leaks no extra information about $\tilde{f}^* \cdot \tilde{f}^*$ [60].

Note that although $\tilde{g}^*$ is not a multilinear polynomial, it could be rewritten as $\tilde{g}^*(X) := \langle \mathbf{v}^*, \mathbf{X}^* \rangle$, where $\mathbf{v}^* = (v_0^*, (v_{1,j}^*)_{j \in [6]}, (v_{2,j}^*)_{j \in [6]}, \dots, (v_{\ell,j}^*)_{j \in [6]})$, $\mathbf{X}^* = (1, (X_1^j)_{j \in [6]}, (X_2^j)_{j \in [6]}, \dots, (X_{\ell}^j)_{j \in [6]})$. We then abuse the notations for multilinear polynomials and define $C_{g^*} \leftarrow \text{PCom}(\tilde{g}^*) = \mathbf{g}^{\mathbf{v}^*} Y^{\mathbf{r}_v}$ where $\mathbf{g} \in \mathbb{G}^{1+6\ell}$, $Y \in \mathbb{G}$, $\mathbf{r}_v \in \mathbb{F}_p$ are uniformly random. We define the Eval protocol to prove that $C_{g^*} = \mathbf{g}^{\mathbf{v}^*} Y^{\mathbf{r}_v} \wedge v_{g^*} = \langle \mathbf{v}^*, \mathbf{r}_v \rangle, (\mathbf{r}^*, 0)$ by using variants of Bulletproofs [1, 5, 56].

Making the Eval protocol zero-knowledge. In the Eval protocol, the prover and verifier engage in variants of Bulletproofs [1, 5, 56] to prove an inner product relation where one vector is public. Following [1], zero-knowledge is added by performing a standard Σ -protocol beforehand. The full specification of the zero-knowledge Eval protocol is deferred to Appendix F.

⁶As $\tilde{f}, \tilde{R}$ are multilinear polynomials represented with interpolation, for convenient implementation, we set $\tilde{Z}$ to be the product of two multilinear polynomials $\prod_{i=1}^{\ell} X_i, \prod_{i=1}^{\ell} (1 - X_i)$ that are both represented with interpolation.

4.1.3 Verifying the sum. In sum-check protocol $c \leftarrow \langle \mathcal{P}, \mathcal{V}(\mathbf{r}) \rangle (\tilde{f}^* \cdot \tilde{f}^* + \rho \cdot \tilde{g}^*, \ell, (6, 6, \dots, 6), u + \rho u^*)$, the verifier checks an equation involving $u + \rho u^*$ in the first round (see Appendix E for details). However, in relation $\mathcal{R}_{ss}$, u is committed as $\hat{B}$ and thus the verifier cannot compute $u + \rho u^*$ directly. To address this, we let the prover send $u + \rho u^*$ to the verifier and use a Σ protocol to prove it is well-formed before executing the sum-check. Specifically, instead of sending u^* , the prover computes and sends $\hat{C} := g^{u^*} h^{r^*}$ to the verifier where $r^* \xleftarrow{\$} \mathbb{F}_p$. After receiving the challenge ρ , the prover computes and sends $z := u + \rho u^*, z' := r + \rho r^*$. The verifier checks $\hat{B} \cdot \hat{C}^{\rho} = g^z h^{z'}$. By the special honest-verifier zero-knowledge and special soundness of Σ protocol, this modification ensures a ZKAoK for relation $\mathcal{R}_{ss}$.

4.2 Formal description

Public parameters: $(n \in \mathbb{N}, \mathbf{g}, Y \in \mathbb{G}^n, g, h \in \mathbb{G})$.

Instance: $(B, \hat{B} \in \mathbb{G})$. **Witness:** $(v, \mathbf{x} \in \mathbb{F}_p^n, u, \mathbf{r} \in \mathbb{F}_p)$.

- $\mathcal{P}$ samples two random polynomials $\tilde{g}^*(X) := v_0^* + \tilde{g}_1^*(X_1) + \tilde{g}_2^*(X_2) + \dots + \tilde{g}_{\ell}^*(X_{\ell})$, $\tilde{R}(X) := \langle (w, \mathbf{0}_{n-1}), (\tilde{e}q(X, \mathbf{y}))_{\mathbf{y} \in \{0,1\}^{\ell}} \rangle$ and commits to them as $C_{g^*} \leftarrow \text{PCom}(\tilde{g}^*)$, $C_R \leftarrow \text{PCom}(\tilde{R})$, respectively. $\mathcal{P}$ computes $u^* := \sum_{\mathbf{b} \in \{0,1\}^{\ell}} \tilde{g}^*(\mathbf{b})$ and commits to it as $\hat{C} := g^{u^*} h^{r^*}$ where $r^* \xleftarrow{\$} \mathbb{F}_p$. $\mathcal{P}$ sends $C_{g^*}, C_R, \hat{C}$ to $\mathcal{V}$.
- $\mathcal{V}$ samples and sends $\rho \xleftarrow{\$} \mathbb{F}_p^*$ to $\mathcal{P}$.
- $\mathcal{P}$ computes and sends $z := u + \rho u^*, z' := r + \rho r^*$ to $\mathcal{V}$.
- $\mathcal{V}$ checks $\hat{B} \cdot \hat{C}^{\rho} = g^z h^{z'}$.
- $\mathcal{P}$ and $\mathcal{V}$ execute the sum-check protocol $c \leftarrow \langle \mathcal{P}, \mathcal{V}(\mathbf{r}) \rangle (\tilde{f}^* \cdot \tilde{f}^* + \rho \cdot \tilde{g}^*, \ell, (6, 6, \dots, 6), z)$ where $\mathbf{r} \xleftarrow{\$} \mathbb{F}_p^{\ell} \setminus \{0, 1\}^{\ell}$, $\tilde{f}^*(X) = \tilde{f}(X) + \tilde{R}(X) \cdot \tilde{Z}(X)$.
- $\mathcal{P}$ and $\mathcal{V}$ execute protocols $\text{Eval}(\text{pp}_{\text{PCS}}, \ell, B \cdot C_R^{\tilde{Z}(\mathbf{r})}, \mathbf{r}, v_{f^*}; \tilde{f}(X) + \tilde{R}(X) \cdot \tilde{Z}(\mathbf{r}))$ and $\text{Eval}(\text{pp}_{\text{PCS}}, \ell, C_{g^*}, \mathbf{r}, v_{g^*}; \tilde{g}^*)$ of the PCS to let $\mathcal{V}$ obtain $v_{f^*} = \tilde{f}^*(\mathbf{r})$, $v_{g^*} = \tilde{g}^*(\mathbf{r})$, respectively.
- $\mathcal{V}$ checks $c = v_{f^*} \cdot v_{f^*} + \rho \cdot v_{g^*}$.

THEOREM 4.1. *The protocol described in Section 4.2 is an interactive ZKAoK for relation $\mathcal{R}_{ss}$ under the discrete logarithm assumption. It has perfect completeness, computational knowledge soundness and perfect honest-verifier zero-knowledge.*

PROOF SKETCH. Completeness follows from the completeness of the Σ protocol, sum-check and PCS. For knowledge soundness, since PCS is extractable, there exists an extractor $\mathcal{E}_{\text{PCS}}$ to extract a polynomial $\tilde{f}^*$ such that $B \cdot C_R^{\tilde{Z}(\mathbf{r})} = \text{PCom}(\tilde{f}^*)$. Repeating the extraction with two different challenges $\mathbf{r}, \mathbf{r}'$, we could extract $(v, \mathbf{x})$ such that $B = g^v Y^{\mathbf{x}}$. Since the Σ protocol is special sound, there exists an extractor $\mathcal{E}_{\Sigma}$ to extract (u, r) such that $\hat{B} = g^u h^r$. Then combining the soundness of the sum-check protocol, we can deduce that $u = \langle v, v \rangle$. Honest-verifier zero-knowledge follows from the hiding of PCS, zero-knowledge of the Σ protocol, the sum-check, and Eval protocol. The formal proof is deferred to Appendix G. $\square$

Efficiency analysis. The protocol contains roughly a basic Σ protocol for proving knowledge of Pedersen commitment openings, a sum-check protocol, and two Eval protocols of PCS. The basic Σ

protocol introduces constant complexity. For the sum-check protocol, according to [8, 60], the prover's work is dominated by $O(n)$ $\mathbb{F}_p$ operations, the proof includes $O(\log n)$ $\mathbb{F}_p$ elements, and the verifier's work is dominated by $O(\log n)$ $\mathbb{F}_p$ operations. For the Eval protocol, according to [1, 5], the prover's work is dominated by $O(n)$ $\mathbb{G}$ exponentiations and $O(n)$ $\mathbb{F}_p$ operations, the proof includes $O(\log n)$ $\mathbb{G}$ elements, and the verifier's work is dominated by $O(n)$ $\mathbb{G}$ exponentiations and $O(n)$ $\mathbb{F}_p$ operations. In summary, the protocol for $\mathcal{R}_{ss}$ introduces $O(n)$ prover complexity, $O(\log n)$ communication complexity and $O(n)$ verifier complexity.

In more detail, the proof consists of C_{g^*} , C_R , $\hat{C}$ ($3 \mathbb{G}$ elements), z , z' ($2 \mathbb{F}_p$ elements), the proof in the sum-check protocol ($7 \log n \mathbb{F}_p$ elements) and the proof in the Eval protocols ($2 \log 2n + 2 \log(6 \log n + 2) - 6 \mathbb{G}$ elements, $12 \mathbb{F}_p$ elements).

We compare our ZKAoK with general-purpose zkSNARKs in Table 2. General-purpose zkSNARKs are typically designed for circuits over fields. To prove the sum-of-square relation, they must encode group operations (e.g., $\hat{B} = g^u h^r$) into arithmetic circuits, often requiring millions of gates (see circom-ecdsa benchmarks⁷). The complexity of any zkSNARK is tied to circuit size $|C|$, which can be millions of times larger than the parameter size n in the sum-of-square relation.

Table 2: Complexity comparisons among ZKAoKs for the sum-of-square relation

Protocol	Prover	Communication	Verifier	Setup
Groth16 [28]	$O(C \log C)$	$O(1)$	$O(1)$	trusted
Aurora [3]	$O(C \log C)$	$O(\log^2 C)$	$O(C)$	transparent
Bulletproofs [5]	$O(C)$	$O(\log C)$	$O(C)$	transparent
Marlin [9]	$O(C \log C)$	$O(1)$	$O(1)$	universally updatable
Spartan [52]	$O(C)$	$O(\sqrt{ C })$	$O(\sqrt{ C })$	transparent
Brakedown [26]	$O(C)$	$O(\sqrt{ C })$	$O(C)$	transparent
HyperPlonk [8]	$O(C)$	$O(\log C)$	$O(\log C)$	universally updatable
Ours	$O(n)$	$O(\log n)$	$O(n)$	transparent

Notes: $|C|$ denotes the size of the circuit expressing the sum-of-square relation. n is the parameter size in the sum-of-square relation.

5 An aggregated range proof with different bases and ranges

In Section 5.1, we define an aggregated range relation and explain how to reduce it. In Section 5.2, we describe our techniques for transforming range constraints into polynomial constraints and explain how to prove them. In Section 5.3, we introduce an optimized inner product argument to reduce the communication complexity from linear to logarithmic. In Section 5.4, we combine Section 5.2, 5.3 and formally present the protocol with security proof and efficiency analysis.

5.1 Relation reduction

An aggregated range proof allows a single prover to prove that multiple committed values lie within specified ranges simultaneously. Most existing protocols only consider cases where the values are committed with the same key and within the same range. We propose VarRange, an aggregated range proof that supports different bases and ranges. Formally, VarRange is for relation $\mathcal{R}_{ar} := \{(n, s, t \in \mathbb{N}, g, h \in \mathbb{G}^{n+1}, B \in \mathbb{G}^{n+1}; v, x \in \mathbb{F}_p^{n+1}) : \text{for } j \in [n+1], B_j = g_j^{v_j} h_j^{x_j} \wedge v_1, \dots, v_n \in [-s, s] \wedge v_{n+1} \in [0, t]\}$. We provide an overview in Figure 2.

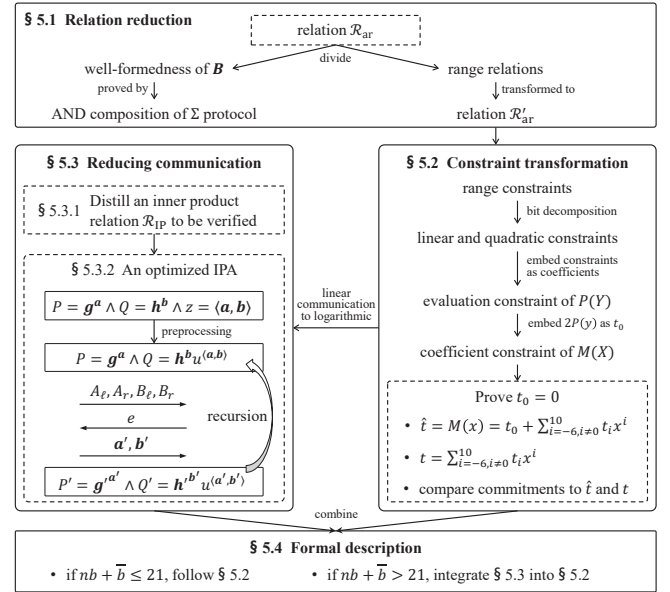


Figure 2: Overview of VarRange

We divide VarRange into two subprotocols. The first subprotocol proves the correct form of B , and the second proves the range relations. Specifically, the first subprotocol is an AND composition of Σ protocol to prove knowledge of v, x such that $B_j = g_j^{v_j} h_j^{x_j}$ for $j \in [n+1]$ (see Appendix H for details), which features $O(n)$ prover, communication and verifier complexities with small coefficients. In QV-net, since relations $\mathcal{R}_{eq}$ and $\mathcal{R}_{ss}$ already ensure the correct form of $\{B_{i,j}\}_{j \in [n_c]}, B_i$, the first subprotocol is not needed when proving $\mathcal{R}_{ar}$. We now focus on the second subprotocol for the range relations, assuming B is well-formed.

We equivalently transform the range relations by leveraging the homomorphism of the Pedersen commitment. To prove $v_j \in [-s, s]$, it is equivalent to proving $v_j + s \in [0, 2s]$. According to [7], for a value b such that $2^{b-1} \leq 2s < 2^b$, it is equivalent to proving $v_j + s \in [0, 2^b] \wedge 2s - (v_j + s) \in [0, 2^b]$. Define a value $\bar{b}$ such that $2^{\bar{b}-1} \leq t < 2^{\bar{b}}$. Since the Pedersen commitment is homomorphic, to prove relation $\mathcal{R}_{ar}$, it suffices to prove the relation $\mathcal{R}'_{ar} := \{(n, b, \bar{b} \in \mathbb{N}, g, h \in \mathbb{G}^{n+1}, C, \hat{C} \in \mathbb{G}^{n+1}; v, \hat{v}, x, \hat{x} \in \mathbb{F}_p^{n+1}) : \text{for } j \in [n+1], C_j = g_j^{v_j} h_j^{x_j} \wedge \hat{C}_j = g_j^{\hat{v}_j} h_j^{\hat{x}_j} \wedge v_1, \hat{v}_1, \dots, v_n, \hat{v}_n \in [0, 2^b] \wedge v_{n+1}, \hat{v}_{n+1} \in [0, 2^{\bar{b}}]\}$, on input $(n, b, \bar{b}, g, h, (B_1 \cdot g_1^s, \dots, B_n \cdot g_n^s, B_{n+1}), (B_1^{-1} \cdot g_1^s, \dots, B_n^{-1} \cdot$

⁷<https://github.com/0xPARC/circom-ecdsa>

$g_n^s, B_{n+1}^{-1} \cdot g_{n+1}^t$; $(v_1+s, \dots, v_n+s, v_{n+1}), (s-v_1, \dots, s-v_n, t-v_{n+1}), \mathbf{x}, -1 \cdot \mathbf{x}$). Hence, we next propose a protocol for relation $\mathcal{R}'_{\text{ar}}$.

Our protocol adapts range proofs from [16, 63]. Daza et al. [16] proposed a method to transform range constraints and constructed a range proof using homomorphic commitments. Zhou et al. [63] improved the efficiency by reducing constraints. However, neither of them supports aggregation. Applying the aggregation techniques from Bulletproofs [5] to Daza et al.'s range proof would require a complete redesign with substantial overhead. Moreover, Zhou et al.'s range proof is inherently incapable of supporting aggregation with different bases.

5.2 Constraint transformation

Let $\mathbf{a}, \hat{\mathbf{a}} \in \{0, 1\}^{nb+\bar{b}}$ denote the concatenation of bit decomposition of $(v_1, \dots, v_n, v_{n+1}), (\hat{v}_1, \dots, \hat{v}_n, \hat{v}_{n+1})$ respectively, and $\mathbf{b} := \mathbf{a} - \mathbf{1}_{nb+\bar{b}}, \hat{\mathbf{b}} := \hat{\mathbf{a}} - \mathbf{1}_{nb+\bar{b}} \in \mathbb{F}_p^{nb+\bar{b}}$, where $\mathbf{1}_{nb+\bar{b}}$ is a vector of length $nb+\bar{b}$ with all elements equal to 1. Let $\mathbf{c} := ((v_1, x_1, \mathbf{0}_{b-2}), \dots, (v_n, x_n, \mathbf{0}_{b-2}), (v_{n+1}, x_{n+1}, \mathbf{0}_{b-2})) \in \mathbb{F}_p^{nb+\bar{b}}$, $\hat{\mathbf{c}} := ((\hat{v}_1, \hat{x}_1, \mathbf{0}_{b-2}), \dots, (\hat{v}_n, \hat{x}_n, \mathbf{0}_{b-2}), (\hat{v}_{n+1}, \hat{x}_{n+1}, \mathbf{0}_{b-2})) \in \mathbb{F}_p^{nb+\bar{b}}$, where $\mathbf{0}_{b-2}$ is a zero vector of length $b-2$ (similarly $\mathbf{0}_{\bar{b}-2}$). Then to prove $v_j, \hat{v}_j \in [0, 2^b]$ for $j \in [n]$ and $v_{n+1}, \hat{v}_{n+1} \in [0, 2^{\bar{b}}]$, it is equivalent to proving⁸

$$\begin{aligned} \mathbf{b} &= \mathbf{a} - \mathbf{1}_{nb+\bar{b}} \wedge \hat{\mathbf{b}} = \hat{\mathbf{a}} - \mathbf{1}_{nb+\bar{b}} \wedge \mathbf{a} \circ \mathbf{b} = \mathbf{0}_{nb+\bar{b}} \wedge \hat{\mathbf{a}} \circ \hat{\mathbf{b}} = \mathbf{0}_{nb+\bar{b}} \\ &\text{for } j \in [n+1], \langle \mathbf{a}, \mathbf{r}_j \rangle = \langle \mathbf{c}, \mathbf{s}_j \rangle \wedge \langle \hat{\mathbf{a}}, \mathbf{r}_j \rangle = \langle \hat{\mathbf{c}}, \mathbf{s}_j \rangle, \end{aligned} \quad (1)$$

where $\mathbf{r}_j := (\mathbf{0}_{(j-1)b}, 2^b, \mathbf{0}_{(n-j)b+\bar{b}}), \mathbf{s}_j := (\mathbf{0}_{(j-1)b}, \mathbf{0}^b, \mathbf{0}_{(n-j)b+\bar{b}})$ for $j \in [n]$, $\mathbf{r}_{n+1} := (\mathbf{0}_{nb}, 2^{\bar{b}}), \mathbf{s}_{n+1} := (\mathbf{0}_{nb}, \mathbf{0}^{\bar{b}})$ with 2^b denoting $(2^0, 2^1, \dots, 2^{b-1})$, $\mathbf{0}^b$ denoting $(1, 0^1, \dots, 0^{b-1})$. The Hadamard product ensures that $\mathbf{a}, \hat{\mathbf{a}}$ are indeed bit vectors. The inner product ensures that $\mathbf{a}, \hat{\mathbf{a}}$ are the correct bit decompositions of $(v_1, \dots, v_n, v_{n+1}), (\hat{v}_1, \dots, \hat{v}_n, \hat{v}_{n+1})$, respectively.

We next rely on Lemma 2.4 to prove the constraints in (1). The intuition is that if a predefined polynomial $f(X)$ takes zero at a random x , then—with overwhelming probability—it is a zero polynomial (all coefficients are zero). Thus, we define a polynomial $P(Y)$ to embed each constraint in (1) into a distinct power of Y :

$$\begin{aligned} P(Y) &:= \langle \mathbf{a}, \mathbf{b} \circ Y^{nb+\bar{b}} \rangle + Y^{nb+\bar{b}} \cdot \langle \hat{\mathbf{a}}, \hat{\mathbf{b}} \circ Y^{nb+\bar{b}} \rangle \\ &\quad + Y^{2(nb+\bar{b})} \cdot \langle \mathbf{a} - \mathbf{1}_{nb+\bar{b}}, \mathbf{b}, Y^{nb+\bar{b}} \rangle \\ &\quad + Y^{3(nb+\bar{b})} \cdot \langle \hat{\mathbf{a}} - \mathbf{1}_{nb+\bar{b}}, \hat{\mathbf{b}}, Y^{nb+\bar{b}} \rangle \\ &\quad + Y^{4(nb+\bar{b})} \cdot \left(\sum_{j \in [n+1]} (\langle \mathbf{a}, \mathbf{r}_j \rangle - \langle \mathbf{c}, \mathbf{s}_j \rangle) \cdot Y^{j-1} \right) \\ &\quad + Y^{4(nb+\bar{b})+n+1} \cdot \left(\sum_{j \in [n+1]} (\langle \hat{\mathbf{a}}, \mathbf{r}_j \rangle - \langle \hat{\mathbf{c}}, \mathbf{s}_j \rangle) \cdot Y^{j-1} \right). \end{aligned}$$

Then after the prover committing to $\mathbf{a}, \mathbf{b}, \hat{\mathbf{a}}, \hat{\mathbf{b}}$, to prove Equation (1), it suffices to prove for $y \xleftarrow{\$} \mathbb{F}_p^*$, $P(y) = 0$. However, since $P(Y)$'s coefficients involve quadratic relations among the witnesses, direct proving is challenging. Inspired by [4], we could embed $P(y)$ as a

⁸As an example, if a value v satisfies that $v = a_1 \cdot 2^0 + a_2 \cdot 2^1 + a_3 \cdot 2^2 + a_4 \cdot 2^3$ where $\mathbf{a} = (a_1, \dots, a_4)$ is a bit vector, then v must belong to range $[0, 2^4]$.

coefficient of another univariate polynomial and utilize the homomorphic property of commitments to prove this coefficient is zero. Specifically, we define a Laurent polynomial $M(X)$ as follows:

$$\begin{aligned} L(X) &:= \mathbf{a}X + \mathbf{b}X^{-1} + y^{nb+\bar{b}}\hat{\mathbf{a}}X^2 + \hat{\mathbf{b}}X^{-2} + \mathbf{c}X^3 + \hat{\mathbf{c}}X^4 + \mathbf{d}X^5, \\ R(X) &:= \left(y^{2(nb+\bar{b})} \mathbf{y}^{nb+\bar{b}} + \sum_{j=1}^{n+1} y^{4(nb+\bar{b})+j-1} \mathbf{r}_j \right) X^{-1} \\ &\quad + \left(y^{2(nb+\bar{b})} \mathbf{y}^{nb+\bar{b}} + \sum_{j=1}^{n+1} y^{3(nb+\bar{b})+n+j} \mathbf{r}_j \right) X^{-2} \\ &\quad - y^{2(nb+\bar{b})} \mathbf{y}^{nb+\bar{b}} X - y^{3(nb+\bar{b})} \mathbf{y}^{nb+\bar{b}} X^2 \\ &\quad - \left(\sum_{j=1}^{n+1} y^{4(nb+\bar{b})+j-1} \mathbf{s}_j \right) X^{-3} - \left(\sum_{j=1}^{n+1} y^{4(nb+\bar{b})+n+j} \mathbf{s}_j \right) X^{-4}, \\ M(X) &:= \langle L(X), L(X) \circ \mathbf{y}^{nb+\bar{b}} + 2R(X) \rangle \\ &\quad - 2 \cdot \langle \mathbf{1}_{nb+\bar{b}}, (y^{2(nb+\bar{b})} + y^{3(nb+\bar{b})}) \cdot \mathbf{y}^{nb+\bar{b}} \rangle = t_0 + \sum_{i=-6, i \neq 0}^{10} t_i X^i. \end{aligned}$$

Here $\mathbf{d} \xleftarrow{\$} \mathbb{F}_p^{nb+\bar{b}}$ is a blinding vector sampled by the prover and will be committed before the challenge y . By reorganizing $M(X)$ in terms of its powers of X , we have that the constant coefficient t_0 of $M(X)$ is exactly $2P(y)$. Thus, to prove $P(y) = 0$, it is equivalent to proving that $t_0 = 0$.

The prover and verifier proceed as follows to prove $t_0 = 0$. The prover will commit to the coefficients $\{t_i\}_{i=-6, i \neq 0}^{10}$ separately as $\{T_i\}_{i=-6, i \neq 0}^{10}$. Then for a uniformly random challenge $x \xleftarrow{\$} \mathbb{F}_p^*$, we let the verifier obtain $\hat{t} := M(x) = t_0 + \sum_{i=-6, i \neq 0}^{10} t_i x^i$. If the commitment to $\hat{t}$ equals the commitment to $t := \sum_{i=-6, i \neq 0}^{10} t_i x^i$, by the binding property of the commitment scheme and Lemma 2.4, we can conclude that $t_0 = 0$. Specifically, the prover will reveal $L := L(x)$ to the verifier. The verifier verifies the correctness of L using the homomorphic properties of the Pedersen vector commitment, and then independently computes $\hat{t}$ and its corresponding commitment. Meanwhile, the verifier computes the commitment to t using $\{T_i\}_{i=-6, i \neq 0}^{10}$, leveraging the homomorphic property of the Pedersen commitment. Finally, the verifier checks if the commitment to $\hat{t}$ matches the commitment to t . If they are equal, the verifier can confirm that $t_0 = 0$, which implies that the range constraints are satisfied.

5.3 Reducing the communication

The protocol described in Section 5.2 requires the prover to send vector L to the verifier, which incurs $O(nb+\bar{b})$ communication complexity. Similar to [16, 63], we could use an inner product argument (IPA) to achieve $O(\log(nb+\bar{b}))$ communication complexity.

5.3.1 Invoking IPAs. After receiving L , the verifier performs the following partial operations. Firstly, the verifier uses the commitments to $\mathbf{a}, \mathbf{b}, \hat{\mathbf{a}}, \hat{\mathbf{b}}, \mathbf{c}, \hat{\mathbf{c}}, \mathbf{d}$ to construct a commitment P according to definition of $L(X)$ and verifies $P = \text{Com}(L)$. Secondly, the verifier computes $R := R(x)$ and $\hat{t} := \langle L, L \circ \mathbf{y}^{nb+\bar{b}} + 2R \rangle - 2 \cdot \langle \mathbf{1}_{nb+\bar{b}}, (y^{2(nb+\bar{b})} + y^{3(nb+\bar{b})}) \cdot \mathbf{y}^{nb+\bar{b}} \rangle$. To invoke IPAs, we let the verifier obtain $\hat{t}$ sent

from the prover and the commitment Q to $L \circ \mathbf{y}^{nb+\bar{b}} + 2R$ by leveraging the homomorphic property of the Pedersen vector commitment. Then the verifier's check can be equivalently modified as checking $P = \text{Com}(L) \wedge Q = \text{Com}(L \circ \mathbf{y}^{nb+\bar{b}} + 2R) \wedge \hat{t} + 2 \cdot \langle \mathbf{1}_{nb+\bar{b}}, (\mathbf{y}^{2(nb+\bar{b})} + \mathbf{y}^{3(nb+\bar{b})}) \cdot \mathbf{y}^{nb+\bar{b}} \rangle = \langle L, L \circ \mathbf{y}^{nb+\bar{b}} + 2R \rangle$, which is captured by the inner product relation $\mathcal{R}_{\text{IP}} := \{(\hat{n} \in \mathbb{N}, \mathbf{g}, \mathbf{h} \in \mathbb{G}^{\hat{n}}, P, Q \in \mathbb{G}, z \in \mathbb{F}_p; \mathbf{a}, \mathbf{b} \in \mathbb{F}_p^{\hat{n}}) : P = \mathbf{g}^{\mathbf{a}} \wedge Q = \mathbf{h}^{\mathbf{b}} \wedge z = \langle \mathbf{a}, \mathbf{b} \rangle\}$. Then the prover no longer needs to send L to the verifier. Instead, it suffices to invoke an IPA for $\mathcal{R}_{\text{IP}}$ to prove the knowledge of L .

5.3.2 Optimizing IPAs. We adapt an existing IPA for lower complexity. Bootle et al. [4] proposed an IPA with $6 \log \hat{n}$ communication complexity by using recursive techniques. Later, Zhang et al. [62] reduced the communication complexity to $4 \log \hat{n}$ by using a random challenge to combine the last two constraints in $\mathcal{R}_{\text{IP}}$. However, the computation complexity remains undesirable. We slightly modify their protocol, reducing the computation complexity by approximately half.

We briefly review the IPA from Zhang et al. [62]. With a uniformly random element $u \in \mathbb{G}$, they first combine constraints $Q = \mathbf{h}^{\mathbf{b}} \wedge z = \langle \mathbf{a}, \mathbf{b} \rangle$ into a single constraint $Q = \mathbf{h}^{\mathbf{b}} u^{\langle \mathbf{a}, \mathbf{b} \rangle}$ using a random challenge, and then rely on recursive techniques to achieve $4 \log \hat{n}$ communication complexity. In the recursion process, they first split every vector into 2 subvectors, i.e., $\mathbf{g} = (\mathbf{g}_\ell, \mathbf{g}_r), \mathbf{h} = (\mathbf{h}_\ell, \mathbf{h}_r), \mathbf{a} = (\mathbf{a}_\ell, \mathbf{a}_r), \mathbf{b} = (\mathbf{b}_\ell, \mathbf{b}_r)$, and then let the prover compute and send shifted commitments $A_\ell := \mathbf{g}_r^{\mathbf{a}_\ell}, A_r := \mathbf{g}_\ell^{\mathbf{a}_r}, B_\ell := \mathbf{h}_r^{\mathbf{b}_\ell} \cdot u^{\langle \mathbf{a}_r, \mathbf{b}_\ell \rangle}, B_r := \mathbf{h}_\ell^{\mathbf{b}_r} \cdot u^{\langle \mathbf{a}_\ell, \mathbf{b}_r \rangle}$. After receiving the uniformly random challenge $e \in \mathbb{F}_p$ from the verifier, the prover computes $\mathbf{a}' := e\mathbf{a}_\ell + e^2\mathbf{a}_r, \mathbf{b}' := e^{-1}\mathbf{b}_\ell + e^{-2}\mathbf{b}_r$. Then both parties separately compute reduced commitments $P' := A_\ell^{e^{-1}} P A_r^e, Q' := B_\ell^e Q B_r^{e^{-1}}$ and reduced commitment keys $\mathbf{g}' := \mathbf{g}_\ell^{e^{-1}} \circ \mathbf{g}_r^{e^{-2}}, \mathbf{h}' := \mathbf{h}_\ell^e \circ \mathbf{h}_r^{e^2}$. The prover and verifier recursively repeat the above process to prove the knowledge of $\mathbf{a}', \mathbf{b}'$ such that $P' = \mathbf{g}'^{\mathbf{a}'} \wedge Q' = \mathbf{h}'^{\mathbf{b}'} u^{\langle \mathbf{a}', \mathbf{b}' \rangle}$ until the vector dimension is a small constant.

In the above recursion process, the computation of $\mathbf{a}', \mathbf{b}'$ requires $2\hat{n}$ field multiplications, and the computation of $\mathbf{g}', \mathbf{h}'$ requires $2\hat{n}$ group exponentiations. We modify the computation method for these new vectors, such that the computation complexity is halved. Specifically, we let $\mathbf{a}' := \mathbf{a}_\ell + e\mathbf{a}_r, \mathbf{b}' := e\mathbf{b}_\ell + \mathbf{b}_r, \mathbf{g}' := \mathbf{g}_\ell^e \circ \mathbf{g}_r, \mathbf{h}' := \mathbf{h}_\ell \circ \mathbf{h}_r^e, P' := A_\ell P^e A_r^e, Q' := B_\ell^e Q^e B_r$. We have that $P' = \mathbf{g}'^{\mathbf{a}'} \wedge Q' = \mathbf{h}'^{\mathbf{b}'} u^{\langle \mathbf{a}', \mathbf{b}' \rangle}$ still holds. Now, the computations of $\mathbf{a}', \mathbf{b}'$ and $\mathbf{g}', \mathbf{h}'$ would require only $\hat{n}$ field multiplications and $\hat{n}$ group exponentiations, respectively. We formally present our IPA for relation $\mathcal{R}_{\text{IP}}$ below.

Public parameters: $(\hat{n} \in \mathbb{N}, \mathbf{g}, \mathbf{h} \in \mathbb{G}^{\hat{n}})$.

Instance: $(P, Q \in \mathbb{G}, z \in \mathbb{F}_p)$. **Witness:** $(\mathbf{a}, \mathbf{b} \in \mathbb{F}_p^{\hat{n}})$.

- $\mathcal{V}$ samples and sends $c \xleftarrow{\$} \mathbb{F}_p$.
- Let $q \in \mathbb{G}$ be a uniformly random element. $\mathcal{P}$ and $\mathcal{V}$ run the following process to prove the relation defined below on input $(\hat{n}, \mathbf{g}, \mathbf{h}, q^c, P, Q \cdot q^{cz}; \mathbf{a}, \mathbf{b})$.

$\{(\hat{n} \in \mathbb{N}, \mathbf{g}, \mathbf{h} \in \mathbb{G}^{\hat{n}}, u, P, Q \in \mathbb{G}; \mathbf{a}, \mathbf{b} \in \mathbb{F}_p^{\hat{n}}) : P = \mathbf{g}^{\mathbf{a}} \wedge Q = \mathbf{h}^{\mathbf{b}} u^{\langle \mathbf{a}, \mathbf{b} \rangle}\}$.

1. If $\hat{n} \leq 4$:
 - $\mathcal{P}$ sends $\mathbf{a}, \mathbf{b}$.

- $\mathcal{V}$ checks $P = \mathbf{g}^{\mathbf{a}}, Q = \mathbf{h}^{\mathbf{b}} u^{\langle \mathbf{a}, \mathbf{b} \rangle}$. If both equations hold, $\mathcal{V}$ accepts; otherwise, $\mathcal{V}$ rejects.
- 2. Else:
 - Split vectors as $\mathbf{g} = (\mathbf{g}_\ell, \mathbf{g}_r), \mathbf{h} = (\mathbf{h}_\ell, \mathbf{h}_r), \mathbf{a} = (\mathbf{a}_\ell, \mathbf{a}_r), \mathbf{b} = (\mathbf{b}_\ell, \mathbf{b}_r)$.
 - $\mathcal{P}$ computes and sends $A_\ell := \mathbf{g}_r^{\mathbf{a}_\ell}, A_r := \mathbf{g}_\ell^{\mathbf{a}_r}, B_\ell := \mathbf{h}_r^{\mathbf{b}_\ell} u^{\langle \mathbf{a}_r, \mathbf{b}_\ell \rangle}, B_r := \mathbf{h}_\ell^{\mathbf{b}_r} u^{\langle \mathbf{a}_\ell, \mathbf{b}_r \rangle}$.
 - $\mathcal{V}$ samples and sends $e \xleftarrow{\$} \mathbb{F}_p$. $\mathcal{V}$ then computes $P' := A_\ell P^e A_r^e, Q' := B_\ell^e Q^e B_r, \mathbf{g}' := \mathbf{g}_\ell^e \circ \mathbf{g}_r, \mathbf{h}' := \mathbf{h}_\ell \circ \mathbf{h}_r^e$.
 - $\mathcal{P}$ computes $\mathbf{a}' := \mathbf{a}_\ell + e \cdot \mathbf{a}_r, \mathbf{b}' := e \cdot \mathbf{b}_\ell + \mathbf{b}_r, \mathbf{g}' := \mathbf{g}_\ell^e \circ \mathbf{g}_r, \mathbf{h}' := \mathbf{h}_\ell \circ \mathbf{h}_r^e$.
 - $\mathcal{P}$ and $\mathcal{V}$ recursively repeat the above process from Step 1 with updated input $(\hat{n}/2, \mathbf{g}', \mathbf{h}', u, P', Q'; \mathbf{a}', \mathbf{b}')$.

THEOREM 5.1. *The protocol described in Section 5.3 is an interactive argument of knowledge for relation $\mathcal{R}_{\text{IP}}$ under the discrete logarithm assumption. It has perfect completeness and computational witness-extended emulation.*

PROOF. The proof basically follows from [62]. Since we only modify the computation methods for $\mathbf{a}', \mathbf{b}', \mathbf{g}', \mathbf{h}', P', Q'$, it suffices to prove that the modifications do not affect the security proof in [62]. For perfect completeness, we have shown that the modifications do not affect the verification equations. For computational witness-extended emulation, Zhang et al. [62] construct an extractor to extract a valid witness for their IPA, based on the fact that when computing P' (similarly Q'), the elements A_ℓ, P, A_r are raised to different powers of e . By rewinding the prover with the same A_ℓ, P, A_r but different challenges e , and taking appropriate linear combinations of the verification equations, the extractor can extract a valid witness. As our modifications do not violate the above fact, their extractor can be naturally applied to our protocol. $\square$

Efficiency analysis. The proof consists of shifted commitments A_ℓ, A_r, B_ℓ, B_r in $\log \hat{n} - 2$ rounds and $\mathbf{a}, \mathbf{b}$ with dimension 4, which sum up to $4 \log \hat{n} - 8 \mathbb{G}$ elements and $8 \mathbb{F}_p$ elements. The prover's work is dominated by computing shifted commitments A_ℓ, A_r, B_ℓ, B_r and reduced $\mathbf{a}', \mathbf{b}', \mathbf{g}', \mathbf{h}'$ in $\log \hat{n} - 2$ rounds, which need $O(\hat{n}) \mathbb{G}$ exponentiations and $O(\hat{n}) \mathbb{F}_p$ operations. The verifier's work is dominated by computing P', Q' and reduced $\mathbf{g}', \mathbf{h}'$ in $\log \hat{n} - 2$ rounds, which need $O(\hat{n}) \mathbb{G}$ exponentiations.

5.4 Formal description

Theoretically, using IPAs reduces the asymptotic communication complexity from $O(nb + \bar{b})$ to $O(\log(nb + \bar{b}))$. However, when $nb + \bar{b}$ is small, the concrete complexity is lower without IPAs. Below, we formally present a protocol for relation $\mathcal{R}'_{\text{a}}$, depending on the concrete value of $nb + \bar{b}$ for lower communication complexity.

Public parameters: $(n, b, \bar{b} \in \mathbb{N}, \mathbf{g}, \mathbf{h} \in \mathbb{G}^{n+1})$.

Instance: $(C, \hat{C} \in \mathbb{G}^{n+1})$. **Witness:** $(v, \hat{v}, \mathbf{x}, \hat{\mathbf{x}} \in \mathbb{F}_p^{n+1})$.

- $\mathcal{P}$ samples $\rho_a, \rho_{\hat{a}}, \rho_b, \rho_{\hat{b}}, \rho_d \xleftarrow{\$} \mathbb{F}_p, \mathbf{d} \xleftarrow{\$} \mathbb{F}_p^{nb+\bar{b}}$. For uniformly random elements $\mathbf{h} \in \mathbb{G}, \{\mathbf{u}_j \in \mathbb{G}^{b-2}\}_{j \in [n]}, \mathbf{u}_{n+1} \in \mathbb{G}^{\bar{b}-2}$, let $\mathbf{g} := (g_1, h_1, \mathbf{u}_1, \dots, g_n, h_n, \mathbf{u}_n, g_{n+1}, h_{n+1}, \mathbf{u}_{n+1}) \in \mathbb{G}^{nb+\bar{b}}$. $\mathcal{P}$ computes and sends $A := \mathbf{g}^{\mathbf{a}} \mathbf{h}^{\rho_a}, \hat{A} := \mathbf{g}^{\hat{\mathbf{a}}} \mathbf{h}^{\rho_{\hat{a}}}, B := \mathbf{g}^{\mathbf{b}} \mathbf{h}^{\rho_b}, \hat{B} := \mathbf{g}^{\hat{\mathbf{b}}} \mathbf{h}^{\rho_{\hat{b}}}, D := \mathbf{g}^{\mathbf{d}} \mathbf{h}^{\rho_d}$.

- $\mathcal{V}$ samples and sends $y \xleftarrow{\$} \mathbb{F}_p^*$.
- $\mathcal{P}$ defines and computes polynomials $L(X), R(X), M(X)$.
For $i \in \{-6, \dots, -1, 1, \dots, 10\}$, $\mathcal{P}$ samples $\tau_i \xleftarrow{\$} \mathbb{F}_p$, then computes and sends $T_i := g_1^{\tau_i} h^{\tau_i}$.
- $\mathcal{V}$ samples and sends $x \xleftarrow{\$} \mathbb{F}_p^*$.
- If $nb + \bar{b} \leq 21$:
 - $\mathcal{P}$ computes and sends $L := L(x)$, $\tau := \sum_{i=-6, i \neq 0}^{10} \tau_i \cdot x^i$,
 $\rho := \rho_a \cdot x + \rho_b \cdot x^{-1} + \rho_{\hat{a}} \cdot y^{nb+\bar{b}} \cdot x^2 + \rho_{\hat{b}} \cdot x^{-2} + \rho_d \cdot x^5$.
 - $\mathcal{V}$ computes $R := R(x)$, $\hat{t} := M(x)$. $\mathcal{V}$ checks $\mathbf{g}^L = A^x \cdot B^{x^{-1}} \cdot \hat{A} y^{nb+\bar{b}} \cdot x^2 \cdot \hat{B} x^{-2} \cdot \left(\prod_{j \in [n+1]} C_j \right)^{x^3} \cdot \left(\prod_{j \in [n+1]} \hat{C}_j \right)^{x^4} \cdot D^{x^5} \cdot h^{-\rho}$, $g_1^{\hat{t}} h^{\tau} = \prod_{i=-6, i \neq 0}^{10} T_i^{x^i}$. If both equations hold, $\mathcal{V}$ accepts; otherwise, $\mathcal{V}$ rejects.
- Else⁹:
 - $\mathcal{P}$ computes $L := L(x)$, $R := R(x)$, $\hat{t} := M(x)$, $\tau := \sum_{i=-6, i \neq 0}^{10} \tau_i \cdot x^i$, $\rho := \rho_a \cdot x + \rho_b \cdot x^{-1} + \rho_{\hat{a}} \cdot y^{nb+\bar{b}} \cdot x^2 + \rho_{\hat{b}} \cdot x^{-2} + \rho_d \cdot x^5$, $\mathbf{h} := (g_1, g_2^{y^{-1}}, g_3^{y^{-2}}, \dots, g_{nb+\bar{b}}^{y^{-(nb+\bar{b}-1)}})$. $\mathcal{P}$ sends $\hat{t}, \tau, \rho$.
 - $\mathcal{V}$ checks $g_1^{\hat{t}} h^{\tau} = \prod_{i=-6, i \neq 0}^{10} T_i^{x^i}$, and rejects if it does not hold. $\mathcal{V}$ then computes $R := R(x)$, $\mathbf{h} := (g_1, g_2^{y^{-1}}, g_3^{y^{-2}}, \dots, g_{nb+\bar{b}}^{y^{-(nb+\bar{b}-1)}})$, $P := A^x \cdot B^{x^{-1}} \cdot \hat{A} y^{nb+\bar{b}} \cdot x^2 \cdot \hat{B} x^{-2} \cdot \left(\prod_{j \in [n+1]} C_j \right)^{x^3} \cdot \left(\prod_{j \in [n+1]} \hat{C}_j \right)^{x^4} \cdot D^{x^5} \cdot h^{-\rho}$, $Q := P \cdot \mathbf{h}^{2R}$.
 - $\mathcal{P}$ and $\mathcal{V}$ invoke an IPA for relation $\mathcal{R}_{\text{IP}}$ on input $(nb + \bar{b}, \mathbf{g}, \mathbf{h}, P, Q, \hat{t} + 2 \cdot \langle 1_{nb+\bar{b}}, (y^{2(nb+\bar{b}}) + y^{3(nb+\bar{b}}) \cdot y^{nb+\bar{b}}) \rangle; L, L_0 y^{nb+\bar{b}} + 2R)$.

THEOREM 5.2. *The protocol described in Section 5.4 is an interactive ZKAoK for relation $\mathcal{R}'_{\text{ar}}$ under the discrete logarithm assumption. It has perfect completeness, computational witness-extended emulation and perfect special honest-verifier zero-knowledge.*

PROOF SKETCH. Completeness follows from the homomorphism of the Pedersen (vector) commitment and completeness of the IPA. For witness-extended emulation, we construct an extractor $\mathcal{E}$ to extract a valid witness for $\mathcal{R}'_{\text{ar}}$. $\mathcal{E}$ rewinds the prover with $4(nb + \bar{b}) + 2n + 2$ different challenges y and 17 different challenges x , obtaining $(4(nb + \bar{b}) + 2n + 2) \cdot 17$ accepting transcripts. By appropriately combining the equations satisfied by these transcripts, we can compute a valid witness. For special honest-verifier zero-knowledge, we construct a simulator $\mathcal{S}$ that produces a transcript perfectly indistinguishable from a real execution. $\mathcal{S}$ computes challenges using the verifier's randomness, samples some proof elements randomly, and computes others according to verification equations. The formal proof is deferred to Appendix I. $\square$

Efficiency analysis. The efficiency depends on the value of $nb + \bar{b}$. When $nb + \bar{b} \leq 21$, the proof consists of $A, \hat{A}, B, \hat{B}, D, \{T_i\}_{i=-6, i \neq 0}^{10}, L, \tau, \rho$, which sum up to 21 $\mathbb{G}$ elements and $nb + \bar{b} + 2 \mathbb{F}_p$ elements. The prover's work is dominated by computing $A, \hat{A}, B, \hat{B}, D$, the

polynomial $M(X)$, $\{T_i\}_{i=-6, i \neq 0}^{10}$, and L, τ, ρ , which need $O(nb + \bar{b})$ $\mathbb{G}$ exponentiations and $O(nb + \bar{b}) \mathbb{F}_p$ operations. The verifier's work is dominated by computing $R, \hat{t}$ and verifying $\mathbf{g}^L, g_1^{\hat{t}} h^{\tau}$, which need $O(nb + \bar{b})$ $\mathbb{G}$ exponentiations and $O(nb + \bar{b}) \mathbb{F}_p$ operations.

When $nb + \bar{b} > 21$, the proof consists of $A, \hat{A}, B, \hat{B}, D, \{T_i\}_{i=-6, i \neq 0}^{10}, \hat{t}, \tau, \rho$, and the proof in the IPA protocol, which sum up to $4 \log(nb + \bar{b}) + 13 \mathbb{G}$ elements and 11 $\mathbb{F}_p$ elements. The prover's work is dominated by computing $A, \hat{A}, B, \hat{B}, D$, the polynomial $M(X)$, $\{T_i\}_{i=-6, i \neq 0}^{10}, L, R, \hat{t}, \tau, \rho, \mathbf{h}$, and invoking the IPA protocol, which need $O(nb + \bar{b})$ $\mathbb{G}$ exponentiations and $O(nb + \bar{b}) \mathbb{F}_p$ operations. The verifier's work is dominated by computing $R, \mathbf{h}, P, Q$, verifying $g_1^{\hat{t}} h^{\tau}$, and invoking the IPA protocol, which need $O(nb + \bar{b})$ $\mathbb{G}$ exponentiations and $O(nb + \bar{b}) \mathbb{F}_p$ operations.

6 Performance analysis of QV-net

We perform a theoretical analysis and an empirical analysis of QV-net. We consider the computation overhead of executing the Voting and Tallying phases and the storage overhead of a bulletin board.

6.1 Theoretical analysis

We summarize our theoretical performance analysis in Table 3 based on the efficiency analysis of ZKAoKs in previous sections.

Table 3: Computation and storage complexities of QV-net

Module	Complexity	Module	Complexity
Gen $\{y_{i,j}\}_{j \in [n_c]}$	$n_c \mathbb{G}$	Sz $\{y_{i,j}\}_{j \in [n_c]}$	$n_c \mathbb{G}$
Gen $\pi_i^{(\text{dl})}$	$n_c \mathbb{G}, 2n_c \mathbb{F}_p$	Sz $\pi_i^{(\text{dl})}$	$n_c \mathbb{G}, n_c \mathbb{F}_p$
Gen $\{B_{i,j}\}_{j \in [n_c]}, B_i$	$(2n_c + 2) \mathbb{G}$	Sz $\{B_{i,j}\}_{j \in [n_c]}, B_i$	$(n_c + 1) \mathbb{G}$
Gen $\pi_i^{(\text{eq})}$	$3n_c \mathbb{G}, 4n_c \mathbb{F}_p$	Sz $\pi_i^{(\text{eq})}$	$2n_c \mathbb{G}, 2n_c \mathbb{F}_p$
Gen $\pi_i^{(\text{ss})}$	$O(n_c) \mathbb{G}$ $O(n_c) \mathbb{F}_p$	Sz $\pi_i^{(\text{ss})}$	$(2 \log 2n_c - 3 + 2 \log(6 \log n_c + 2)) \mathbb{G}$ $(7 \log n_c + 14) \mathbb{F}_p$
Gen $\pi_i^{(\text{ar})}$	$O(n_c b + \bar{b}) \mathbb{G}$ $O(n_c b + \bar{b}) \mathbb{F}_p$	Sz $\pi_i^{(\text{ar})}$ $(n_c b + \bar{b} \leq 21)$	$21 \mathbb{G}$ $(n_c b + \bar{b} + 2) \mathbb{F}_p$
Gen $\{Y_{i,j}\}_{j \in [n_c]}$	$O(n_v n_c) \mathbb{G}_{\text{mul}}$	Sz $\pi_i^{(\text{ar})}$ $(n_c b + \bar{b} > 21)$	$(4 \log(n_c b + \bar{b}) + 13) \mathbb{G}$ $11 \mathbb{F}_p$
Vfy $\pi_i^{(\text{dl})}$	$2n_c \mathbb{G}$	Vfy $\pi_i^{(\text{ss})}$	$O(n_c) \mathbb{G}, O(n_c) \mathbb{F}_p$
Vfy $\pi_i^{(\text{eq})}$	$5n_c \mathbb{G}$	Vfy $\pi_i^{(\text{ar})}$	$O(n_c b + \bar{b}) \mathbb{G}$ $O(n_c b + \bar{b}) \mathbb{F}_p$
tallying	$O(n_c \sqrt{\text{val}}) \mathbb{G}$		

Notes: Gen denotes generate. Vfy denotes verify. Sz denotes size. n_c, n_v, t_i denotes the number of choices, voters and tokens, respectively; $\mathbb{G}$ denotes group exponentiation or group elements; $\mathbb{G}_{\text{mul}}$ denotes group multiplications; $\mathbb{F}_p$ denotes field operations or field elements; b satisfies $2^{b-1} \leq 2 \lfloor \sqrt{t_i} \rfloor < 2^b$; $\bar{b}$ satisfies $2^{\bar{b}-1} \leq t_i < 2^{\bar{b}}$; val is the maximal value among $\{|\sum_{i \in [n_v]} v_{i,j}|\}_{j \in [n_c]}$.

For computation complexity, all procedures in Voting and Tallying phases incur linear complexity in n_c , implying the number of choices is a key factor in practice. Each voter must generate $\{Y_{i,j}\}_{j \in [n_c]}$ using other voters' public keys, which also incurs linear complexity in n_v . The generation and verification of $\pi_i^{(\text{ar})}$ have linear complexity in $b, \bar{b}$, indicating logarithmic complexity in t_i .

⁹We assume that $nb + \bar{b}$ is a power of 2; otherwise, we can pad vectors to satisfy it.

Thus, the number of tokens has a minimal impact on the two procedures. For tallying, we use Shanks’ baby-step giant-step algorithm, which results in a computation complexity scaling as the square root of the tallying result.

For storage complexity, most components exhibit linear complexity in n_c . However, the proof $\pi_i^{(ss)}$ has logarithmic complexity in n_c . The storage complexity of $\pi_i^{(ar)}$ depends on $n_c \cdot b + \bar{b}$, resulting in either linear or logarithmic complexity with parameters $n_c, b, \bar{b}$. In summary, n_c remains the dominant factor. Although t_i affects the complexity of $\pi_i^{(ar)}$, its impact is relatively minimal.

We provide a comparison among cryptographic quadratic voting schemes in Table 4. The benchmark data is obtained from zkalc¹⁰ using gnark-crypto on AWS EC2 m6i.8xlarge. Since QV-net and [43, 45] differ in settings, a complete comparison is challenging. Here, we primarily focus on comparing the efficiency of ballot generation. Note that the works [43, 45] do not provide actual implementations.

Table 4: Comparisons among cryptographic quadratic voting

Scheme	Setting	Curve	Complexity	Exponentiation efficiency
PR17 [43]	CEN	composite-order curve	unclear	unclear
PR20 [45]	CEN	BN254	$O(n_c b) \mathbb{G}_1$ $O(n_c) \mathbb{G}_2$	$\mathbb{G}_1 : 52\mu s$ $\mathbb{G}_2 : 123\mu s$
QV-net	DEC	Secp256k1	$O(n_c b) \mathbb{G}$	$\mathbb{G} : 75\mu s$

Notes: CEN denotes centralized. DEC denotes decentralized. BN254 offers 100-bit security while Secp256k1 offers 128-bit security. The complexity is for ballot generation and is measured by the number of exponentiations. n_c denotes the number of choices. b denotes the bit length of range.

6.2 Empirical analysis

We built an implementation of QV-net in Rust¹¹. We now present the benchmark configuration and evaluation results.

6.2.1 Benchmark configuration. Without loss of generality, we assume each voter V_i has the same t tokens and allocates them evenly across n_c choices, resulting in $\lfloor t/n_c \rfloor$ votes per choice. Many DAOs launch proposals using quadratic voting, including MetFi DAO, Karmaverse, Pistachio DAO, Gitcoin, and MoonDAO. To evaluate the practicability of QV-net, we set n_v, n_c, t to the maximum values found in related proposals of each DAO and report the corresponding efficiency. Moreover, to verify the theoretical complexity analysis in Table 3, we evaluate the following performance metrics of QV-net, using parameter scales derived from real-world DAO proposals implementing quadratic voting.

Computation overhead in Voting phase and Tallying phase, and storage overhead of bulletin board when varying n_c . Since the computation complexity of all procedures and the storage complexity of all components depend on n_c , we evaluate their practical performance by scaling n_c from 1 to 20 and reporting the resulting growth patterns.

Computation overhead in Tallying phase when varying t .

The parameter t determines the number of votes a voter casts for a choice, directly influencing the tallying efficiency. We analyze its impact by increasing t from 2^0 to 2^{25} .

Computation overhead in Tallying phase when varying n_v .

The parameter n_v determines the total number of votes per choice. We assess its impact on Tallying phase by increasing n_v from 2 to 3900.

Computation overhead for generating $\{Y_{i,j}\}_{j \in [n_c]}$ when varying n_v . The generation of $\{Y_{i,j}\}_{j \in [n_c]}$ has linear complexity in n_v . We evaluate its impact by increasing n_v from 2 to 3900.

Prover time, proof size and verifier time of VarRange when varying t . VarRange’s complexity depends on $b, \bar{b}$, which are decided by t . We assess its impact by varying t from 2^0 to 2^{25} .

Furthermore, since Tallying phase involves computing the discrete logarithm of a public group element, it tends to be the performance bottleneck of QV-net even if using Shanks’ baby-step giant-step algorithm. To evaluate the limits of QV-net, we independently measure the performance of the baby-step giant-step algorithm by arbitrarily increasing the discrete logarithm size until the process is killed.

We conducted the experiments on WSL 2 (Ubuntu 22.04) running on a system with an AMD Ryzen 9 5900X CPU (12-Core/3.7GHz/x64) and 128GB RAM. The peak memory usage remains below 3.3GB. We instantiate the elliptic curve group using the Secp256k1 curve from the ZenGo-X/curv library¹². All storage overhead is manually calculated based on the storage complexity presented in Table 3, using the field and group element sizes of the Secp256k1 curve (32 bytes for $\mathbb{F}_p$ and 33 bytes for $\mathbb{G}$).

6.2.2 Evaluation results.

Performance of QV-net in configurations of real-world DAOs.

In Table 5, we report the performance of QV-net in configurations of real-world DAOs, including generating elements in Round 1 and Round 2 of Voting phase, verifying ZKAoKs, tallying, and the storage size of elements on the bulletin board. The results indicate that QV-net is highly practical.

Most operations require only tens of milliseconds or less. The most time-consuming task is tallying in Gitcoin, taking just 255 ms. The Voting phase in Gitcoin has the highest total cost among all DAOs at approximately 170 ms, with verification operations being the most expensive at around 42 ms. However, these overheads are still minimal in practice. For storage, all sizes remain below 2 KB, with the largest total size in Gitcoin being just 7.69 KB.

It should be noted that the governance tokens of the mentioned DAOs are typically stored with 18 decimal places of precision, meaning that the actual value t in Table 5 should be multiplied by 10^{18} . However, this level of precision makes the tallying procedure extremely expensive, even when using the baby-step giant-step algorithm. To analyze this bottleneck, we conducted an experiment to benchmark the algorithm’s performance, with results detailed in Table 6. A common solution to mitigate this issue is to reduce decimal precision in the token’s smart contracts, either by defining fewer decimals or storing scaled values (e.g., 1,000 or 1,000,000 times the actual value) with a scaling factor for computations.

¹⁰<https://zka.lc/>

¹¹The source code is freely available at <https://zenodo.org/records/15553810>.

¹²<https://github.com/ZenGo-X/curv>

Table 5: Performance of QV-net in configurations of real-world DAOs

Module	MetFi	Karmaverse	Pistachio	Bitcoin	Moon
Gen $\{y_{i,j}\}_{j \in [n_c]}$	0.1173	0.3986	0.1139	0.4236	0.8423
Gen $\pi_i^{(dl)}$	0.1215	0.4099	0.1207	0.4485	0.8977
Gen $\{Y_{i,j}\}_{j \in [n_c]}$	6.2548	0.1804	28.7581	78.4846	0.2236
Gen $\{B_{i,j}\}_{j \in [n_c]}, B_i$	0.1776	0.5087	0.1411	0.5651	0.1463
Gen $\pi_i^{(eq)}$	0.3524	1.2301	0.3572	1.3241	0.2688
Gen $\pi_i^{(ss)}$	8.0058	21.1195	7.9611	21.3339	8.0325
Gen $\pi_i^{(ar)}$	28.9336	30.4093	2.0299	67.5636	19.3513
Vfy $\pi_i^{(dl)}$	0.2332	0.8166	0.2332	0.9253	0.1753
Vfy $\pi_i^{(eq)}$	0.5857	2.0281	0.5867	2.1977	0.4389
Vfy $\pi_i^{(ss)}$	1.8103	4.2271	1.8079	4.3155	1.8343
Vfy $\pi_i^{(ar)}$	15.6869	16.5968	1.6277	34.4716	10.2579
tallying	29.1228	0.9739	28.2271	255.54	12.4573
Sz $\{y_{i,j}\}_{j \in [n_c]}$	132	462	132	495	99
Sz $\pi_i^{(dl)}$	260	910	260	975	195
Sz $\{B_{i,j}\}_{j \in [n_c]}, B_i$	165	495	165	528	132
Sz $\pi_i^{(eq)}$	520	1820	520	1950	390
Sz $\pi_i^{(ss)}$	1259	1905	1259	1905	1259
Sz $\pi_i^{(ar)}$	1705	1705	1045	1837	1573

Notes: Gen denotes generate. Vfy denotes verify. Sz denotes size. All times are in milliseconds. All sizes are in bytes. Parameters (n_c, n_v, t) for these DAOs are (4, 857, 1600000), (14, 9, 200), (4, 3700, 1), (15, 2800, 1200000), (3, 42, 13600000).

Performance of QV-net when varying n_c . In Figure 3, we report the computation and storage overhead of QV-net as n_c varies, with $n_v = 2, t = 100$ fixed. The results demonstrate that the practical efficiency of QV-net aligns closely with the theoretical complexity analysis in Table 3.

Figure 3(a) illustrates that the time of generating $\{y_{i,j}\}_{j \in [n_c]}$ and $\pi_i^{(dl)}$ increases linearly with n_c . Notably, the growth of the former is slightly slower than the latter. When $n_c = 20$, the respective times are approximately 0.56 ms and 0.58 ms.

Figure 3(b) illustrates the growth patterns of generating ballots and proofs in Round 2 of Voting phase. The times required for generating $\{B_{i,j}\}_{j \in [n_c]}, B_i$, and $\pi_i^{(eq)}$ both increase linearly with n_c , ranging from 0.07 ms to 0.7 ms and 0.09 ms to 1.7 ms, respectively. Similarly, the time of generating $\pi_i^{(ss)}$ also grows linearly with n_c , reaching 31 ms when $n_c = 20$. However, due to vector padding to the nearest power of 2 in the ZKAoK protocol (Section 4), sharp increases occur when n_c transitions from one power of 2 to the next. Within each power-of-2 padding range, the time remains nearly constant. The generation time of $\pi_i^{(ar)}$ also follows a linear growth pattern, reaching 35 ms when $n_c = 20$. As vectors of length $n_c \cdot b + \bar{b}$ are padded to the nearest power of 2 in VarRange (Section 5) when

invoking the IPA, sharp increases in time occur when n_c transitions between padding ranges, such as from 5 to 6 or from 11 to 12. Within the same padding range (e.g., n_c from 6 to 11), the overhead from IPA remains constant. However, operations outside of the IPA still exhibit linear complexity, causing the overall time to grow slowly in a linear pattern. We note that the time of generating $\{Y_{i,j}\}_{j \in [n_c]}$ ranges from 0.1 μ s to 0.6 μ s, which is significantly smaller than the other times. Therefore, it is excluded from Figure 3(b).

Figure 3(c) illustrates the growth patterns of verifying ZKAoKs. The verification times for $\pi_i^{(dl)}, \pi_i^{(eq)}, \pi_i^{(ss)}$, and $\pi_i^{(ar)}$ increase linearly with n_c , reaching 1.1 ms, 2.9 ms, 6.2 ms, and 18.9 ms at $n_c = 20$. Due to padding, the verification curves for $\pi_i^{(ss)}$ and $\pi_i^{(ar)}$ exhibit the same pattern as their generation curves shown in Figure 3(b).

Figure 3(d) shows that the tallying time generally increases as n_c grows. In our experiment, we assume that each voter allocates tokens evenly across all choices. As a result, when $n_c = 3$ or $n_c = 4$, each voter casts $\lfloor \sqrt{100/3} \rfloor = \lfloor \sqrt{100/4} \rfloor = 5$ votes per choice. For $n_c = 5$ or $n_c = 6$, each voter casts 4 votes per choice. Similarly, for n_c between 7 and 11, each voter casts 3 votes per choice, and for n_c between 12 and 20, each voter casts 2 votes per choice. Within each range, the tallying time increases linearly with n_c . However, when n_c transitions between ranges, the tallying time may either increase or decrease. This variation arises because, while the number of choices increases, the votes allocated per choice decrease. It should be noted that in practical scenarios, each voter may adopt their own strategy for token allocation. This figure reflects the performance of QV-net under the specific strategy used in our experiment.

Figure 3(e) illustrates the growth patterns of the sizes of elements that are stored on the bulletin board. The sizes of $\{y_{i,j}\}_{j \in [n_c]}, \pi_i^{(dl)}, (\{B_{i,j}\}_{j \in [n_c]}, B_i)$, and $\pi_i^{(eq)}$ grow linearly with n_c , reaching 660 bytes, 1300 bytes, 693 bytes, and 2600 bytes, respectively, when $n_c = 20$. In contrast, the sizes of $\pi_i^{(ss)}$ and $\pi_i^{(ar)}$ grow logarithmically with n_c , reaching 2195 bytes and 1705 bytes, respectively, when $n_c = 20$. Due to padding, the sizes of $\pi_i^{(ss)}$ and $\pi_i^{(ar)}$ remain constant within certain ranges of n_c . It is worth noting that, even within the same padding range, while the times for generating and verifying $\pi_i^{(ar)}$ increase slowly but linearly with n_c , the size of $\pi_i^{(ar)}$ remains constant, as the proof excluding IPA is independent of n_c .

Computation overhead in Tallying phase when varying t, n_v . Figure 3(f) illustrates the growth pattern as t varies, with $n_c = 1$ and $n_v = 2$ fixed, while Figure 3(g) illustrates the pattern as n_v varies, with $n_c = 1$ and $t = 100$ fixed. Generally, the tallying time increases with t or n_v . However, in some cases, a larger t results in a shorter tallying time. This occurs because the baby-step giant-step algorithm's runtime does not always increase with the discrete logarithm. For instance, in Figure 3(f), when $t = 2^{10}$, the giant-step is executed 9 times and the baby-step 8 times, whereas for $t = 2^{11}$, the giant-step is executed 10 times and the baby-step 0 times.

Computation overhead of generating $\{Y_{i,j}\}_{j \in [n_c]}$ when varying n_v . In Figure 3(h), we report the computation overhead of generating $\{Y_{i,j}\}_{j \in [n_c]}$ as n_v varies, with $n_c = 1, t = 2$ fixed. The time increases linearly with n_v , reaching 7.3 ms when $n_v = 3900$.

Performance of VarRange when varying t . In Figure 4, we report the performance of VarRange as t varies, with $n_c = 10$ fixed. The

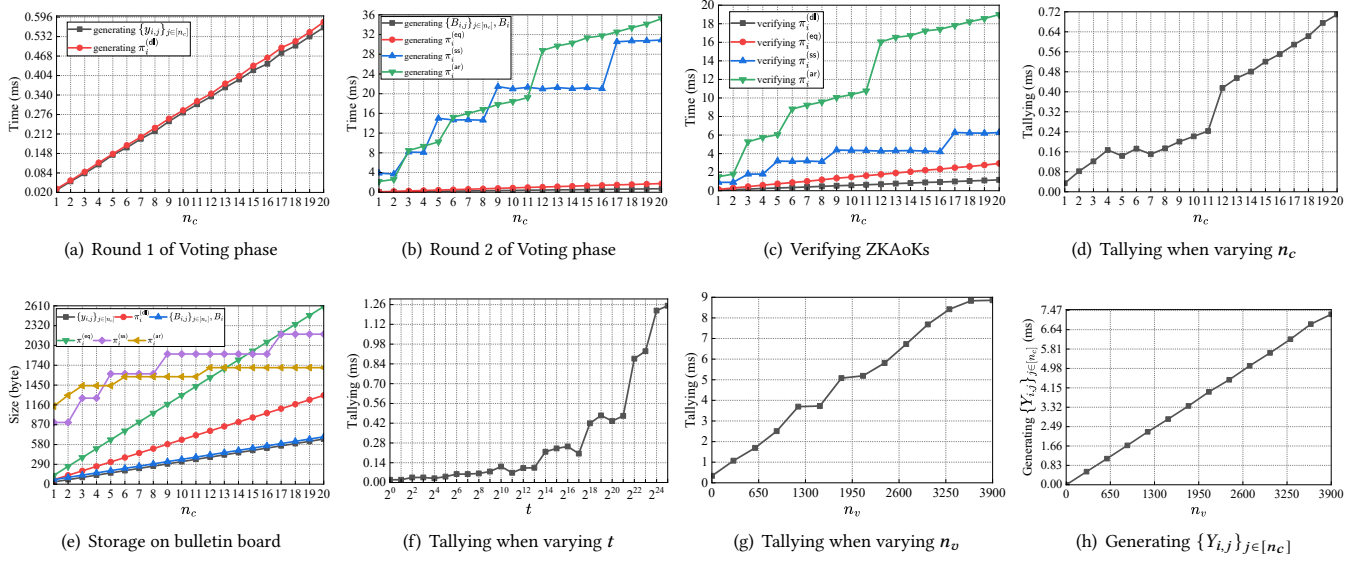
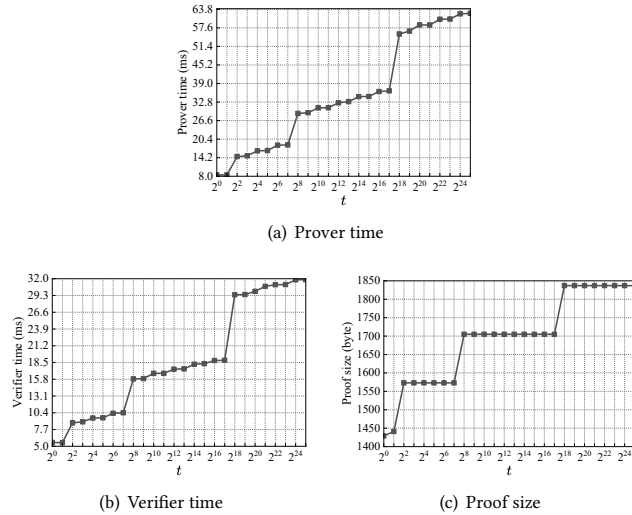


Figure 3: Performance of QV-net when varying parameters

prover time (Figure 4(a)) and verifier time (Figure 4(b)) both grow logarithmically¹³ with t , reaching 62 ms and 32 ms, respectively, at $t = 2^{25}$. The proof size (Figure 4(c)) grows double-logarithmically with t , reaching 1837 bytes at $t = 2^{25}$. Due to padding, sharp increases occur when t transitions between padding ranges, but within the same range, the prover and verifier times grow slowly following a logarithmic pattern, while the proof size remains constant as it is independent of t outside of IPA.

Figure 4: Performance of VarRange when varying t

Performance of baby-step giant-step algorithm. In Table 6, we report the performance of the baby-step giant-step algorithm for

¹³Note that the horizontal axis is plotted on a logarithmic scale.

computing $x \in \mathbb{F}_p$ such that $y = g^x \in \mathbb{G}$, given an upper bound bnd for x . Without loss of generality, we always set x to take the value of bnd . The results show that the algorithm scales to $bnd = 2^{57}$, requiring 5254 seconds. However, in practical scenarios, it is preferable to restrict bnd to values below 2^{36} , where the algorithm completes in just 5 seconds.

Table 6: Performance of baby-step giant-step algorithm

bnd	Time (s)	bnd	Time (s)	bnd	Time (s)	bnd	Time (s)
2^{20}	0.0151	2^{21}	0.0179	2^{22}	0.0307	2^{23}	0.03017
2^{24}	0.0626	2^{25}	0.0797	2^{26}	0.1287	2^{27}	0.1421
2^{28}	0.2656	2^{29}	0.2085	2^{30}	0.5492	2^{31}	0.4555
2^{32}	1.1436	2^{33}	1.0186	2^{34}	2.4224	2^{35}	2.4376
2^{36}	4.8417	2^{37}	6.2402	2^{38}	9.8639	2^{39}	11.5522
2^{40}	20.4595	2^{41}	18.5545	2^{42}	41.9245	2^{43}	43.6534
2^{44}	86.4154	2^{45}	111.7856	2^{46}	176.6134	2^{47}	205.3959
2^{48}	361.6105	2^{49}	329.2993	2^{50}	737.6556	2^{51}	761.5141
2^{52}	1509.8261	2^{53}	1926.5101	2^{54}	3114.7844	2^{55}	3469.1078
2^{56}	6363.2908	2^{57}	5254.1128				

7 Conclusion

Decentralized e-voting is crucial for DAO governance, with quadratic voting mitigating the "whale problem", but existing implementations lack ballot secrecy. In this paper, we propose QV-net,

a decentralized quadratic voting that ensures ballot secrecy, self-tallying, and dispute-freeness. Our proof-of-concept implementation demonstrates its feasibility, with benchmark results showing a tallying time of up to 255 ms, voting time of 170 ms, and ZKAoK verification in 42 ms. The maximum total proof size is just 7.69 KB.

QV-net leverages ZKAoKs to guarantee secrecy and integrity. We formalize key ZKAoK relations, including discrete logarithm knowledge, discrete logarithm equality, sum-of-square, and range proofs. Furthermore, we introduce a succinct ZKAoK for the sum-of-square relation and an aggregated range proof supporting variable bases and ranges, both of independent interest.

Acknowledgment

This work was supported by National Key R&D Program of China under Grant (2022YFB2702702), National Cryptologic Science Fund of China (No. 2025NCSF02022), National Natural Science Foundation of China (62372020, 72031001), Beijing Natural Science Foundation (L222050), and Beijing Advanced Innovation Center for Future Blockchain and Privacy Computing. Feng Hao was supported by the Engineering and Physical Sciences Research Council (EP/T014784/1).

References

- [1] Thomas Attema and Ronald Cramer. 2020. Compressed Σ -Protocol Theory and Practical Application to Plug & Play Secure Algorithmics. In *CRYPTO*. Springer, 513–543.
- [2] Madeline E Bassetti, Gustavo Dias, Daniel L Chen, Alan Morton, and Ritesh Das. 2023. Civicbase: An open-source platform for deploying Quadratic Voting for Survey Research. *AI magazine* 44, 3 (2023), 263–273.
- [3] Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, and Nicholas P. Ward. 2019. Aurora: Transparent Succinct Arguments for R1CS. In *EUROCRYPT*. Springer, 103–128.
- [4] Jonathan Bootle, Andrea Cerulli, Pyrrhos Chaidos, Jens Groth, and Christophe Petit. 2016. Efficient Zero-Knowledge Arguments for Arithmetic Circuits in the Discrete Log Setting. In *EUROCRYPT*. Springer, 327–357.
- [5] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Gregory Maxwell. 2018. Bulletproofs: Short Proofs for Confidential Transactions and More. In *S&P*. IEEE, 315–334.
- [6] Benedikt Bünz, Ben Fisch, and Alan Szepieniec. 2020. Transparent SNARKs from DARK Compilers. In *EUROCRYPT*. Springer, 677–706.
- [7] Jan Camenisch, Rafik Chaabouni, and Abhi Shelat. 2008. Efficient Protocols for Set Membership and Range Proofs. In *ASIACRYPT*. Springer, 234–252.
- [8] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. 2023. HyperPlonk: Plonk with Linear-Time Prover and High-Degree Custom Gates. In *EUROCRYPT*. Springer, 499–530.
- [9] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Psi Vessely, and Nicholas P. Ward. 2020. Marlin: Preprocessing zkSNARKs with Universal and Updatable SRS. In *EUROCRYPT*. Springer, 738–768.
- [10] Miranda Christ, Foteini Baldimtsi, Konstantinos Kryptos Chalkias, Deepak Maram, Arnab Roy, and Joy Wang. 2024. SoK: Zero-Knowledge Range Proofs. Cryptology ePrint Archive, Paper 2024/430. <https://eprint.iacr.org/2024/430>
- [11] HeeWon Chung, Kyoohyung Han, Chanyang Ju, Myungsun Kim, and Jae Hong Seo. 2022. Bulletproofs+: Shorter Proofs for a Privacy-Enhanced Distributed Ledger. *IEEE Access* 10 (2022), 42067–42082.
- [12] Graham Cormode, Michael Mitzenmacher, and Justin Thaler. 2012. Practical verified computation with streaming interactive proofs. In *ITCS*. Association for Computing Machinery, New York, NY, USA, 90–112.
- [13] Geoffroy Couteau, Dahmun Goudarzi, Michael Kloof, and Michael Reichle. 2022. Sharp: Short Relaxed Range Proofs. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 609–622.
- [14] MetFi DAO. 2024. <https://snapshot.box/#/s:metfi.io>
- [15] Pistachio DAO. 2024. <https://snapshot.box/#/s:pistachiodao.eth>
- [16] Vanesa Daza, Carla Ràfols, and Alexandros Zacharakis. 2020. Updateable Inner Product Argument with Logarithmic Verifier and Applications. In *PKC*. Springer, 527–557.
- [17] Nicola Dimitri. 2022. Quadratic voting in blockchain governance. *Information* 13, 6 (2022), 305.
- [18] Qinxu Ding, Daniel Liebau, Zhiguo Wang, and Weibiao Xu. 2023. A survey on decentralized autonomous organizations (DAOs) and their governance. *World Scientific Annual Review of Fintech* 1 (2023), 2350001.
- [19] Liam Eagen, Sanket Kanjalkar, Tim Ruffing, and Jonas Nick. 2024. Bulletproofs++: Next Generation Confidential Transactions via Reciprocal Set Membership Arguments. In *EUROCRYPT*. Springer, 249–279.
- [20] Jon X Eguia, Nicole Immorlica, Katrina Ligett, E Glen Weyl, and Dimitrios Xefteris. 2019. A new consensus protocol: quadratic voting with multiple alternatives. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3319508#paper-citations-widget
- [21] Yixuan Fan, Lei Zhang, Ruiyu Wang, and Muhammad Ali Imran. 2024. Insight Into Voting in DAOs: Conceptual Analysis and a Proposal for Evaluation Framework. *IEEE Network* 38, 3 (2024), 92–99.
- [22] Amos Fiat and Adi Shamir. 1986. How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In *CRYPTO*. Springer, 186–194.
- [23] Rui Gao, Zhiguo Wan, Yuncong Hu, and Huaqun Wang. 2024. A Succinct Range Proof for Polynomial-based Vector Commitment. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 3152–3166.
- [24] Laura Georgescu, James Fox, Anna Gautier, and Michael Wooldridge. 2024. Fixed-budget and Multiple-issue Quadratic Voting. arXiv:2409.06614 <https://arxiv.org/abs/2409.06614>
- [25] Gitcoin. 2024. <https://snapshot.box/#/s:gitcoindao.eth>
- [26] Alexander Golovnev, Jonathan Lee, Srinath T. V. Setty, Justin Thaler, and Riad S. Wahby. 2023. Brakedown: Linear-Time and Field-Agnostic SNARKs for R1CS. In *CRYPTO*. Springer, 193–226.
- [27] Jens Groth. 2004. Efficient Maximal Privacy in Boardroom Voting and Anonymous Broadcast. In *FC*. Springer, 90–104.
- [28] Jens Groth. 2016. On the Size of Pairing-Based Non-interactive Arguments. In *EUROCRYPT*. Springer, 305–326.
- [29] Feng Hao, Peter Ryan, and Piotr Zielinski. 2010. Anonymous voting by two-round public discussion. *IET Information Security* 4, 2 (2010), 62–67.
- [30] Samer Hassan and Primavera De Filippi. 2021. Decentralized Autonomous Organization. *Internet Policy Review* 10, 2 (2021).
- [31] Jun Huang, Debiao He, Mohammad S. Obaidat, Pandi Vijayakumar, Min Luo, and Kim-Kwang Raymond Choo. 2022. The Application of the Blockchain Technology in Voting Systems: A Review. *Comput. Surveys* 54, 3 (2022), 60:1–60:28.
- [32] Karmaverse. 2024. <https://snapshot.box/#/s:karmaverse.eth>
- [33] Dalia Khader, Ben Smyth, Peter Y. A. Ryan, and Feng Hao. 2012. A Fair and Robust Voting System by Broadcast. In *EVOTE*. GI, 285–299.
- [34] Aggelos Kiayias and Moti Yung. 2002. Self-tallying Elections and Perfect Ballot Secrecy. In *PKC*. Springer, 141–158.
- [35] Abhiram Kothapalli and Srinath Setty. 2024. HyperNova: Recursive Arguments for Customizable Constraint Systems. In *CRYPTO*.
- [36] Steven P. Lalley and E. Glen Weyl. 2018. Quadratic voting: How mechanism design can radicalize democracy. In *AEA Papers and Proceedings*.
- [37] Ben Laurence and Itai Sher. 2017. Ethical considerations on quadratic voting. *Public Choice* 172 (2017), 195–222.
- [38] Yannan Li, Willy Susilo, Guomin Yang, Yong Yu, Dongxi Liu, Xiaojiang Du, and Mohsen Guizani. 2022. A Blockchain-Based Self-Tallying Voting Protocol in Decentralized IoT. *IEEE Transactions on Dependable and Secure Computing* 19, 1 (2022), 119–130.
- [39] Carsten Lund, Lance Fortnow, Howard Karloff, and Noam Nisan. 1992. Algebraic methods for interactive proof systems. *J. ACM* 39, 4 (Oct. 1992), 859–868.
- [40] Patrick McCorry, Siamak Shahandashti, and Feng Hao. 2017. A Smart Contract for Boardroom Voting with Maximum Voter Privacy. In *FC*. Springer, 357–375.
- [41] MoonDAO. 2024. <https://snapshot.box/#/s:tomoondao.eth>
- [42] Somnath Panja, Samiran Bag, Feng Hao, and Bimal Kumar Roy. 2020. A Smart Contract System for Decentralized Borda Count Voting. *IEEE Transactions on Engineering Management* 67, 4 (2020), 1323–1339.
- [43] Sunoo Park and Ronald L Rivest. 2017. Towards secure quadratic voting. *Public Choice* 172 (2017), 151–175.
- [44] Torben P. Pedersen. 1991. Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing. In *CRYPTO*. Springer, 129–140.
- [45] Olivier Pereira and Peter B. Rønne. 2019. End-to-End Verifiable Quadratic Voting with Everlasting Privacy. In *FC 2019 International Workshops, VOTING and WTSC*. Springer, 314–329.
- [46] Eric Posner and Glen Weyl. 2013. Quadratic Vote Buying as Efficient Corporate Governance. *Coase-Sandor Institute for Law and Economics Working Paper* 643 (2013).
- [47] Eric A Posner and Nicholas O Stephanopoulos. 2017. Quadratic election law. *Public Choice* 172 (2017), 265–282.
- [48] Eric A Posner and E Glen Weyl. 2017. Quadratic voting and the public good: introduction. *Public Choice* 172 (2017), 1–22.
- [49] David Quarfoot, Douglas von Kohorn, Kevin Slavin, Rory Sutherland, David Goldstein, and Ellen Konar. 2017. Quadratic voting in the wild: real people, real votes. *Public Choice* 172 (2017), 283–303.
- [50] Claus-Peter Schnorr. 1989. Efficient Identification and Signatures for Smart Cards. In *CRYPTO*. Springer, 239–252.

- [51] Jacob T. Schwartz. 1980. Fast Probabilistic Algorithms for Verification of Polynomial Identities. *J. ACM* 27, 4 (Oct. 1980), 701–717.
- [52] Srinath Setty. 2020. Spartan: Efficient and General-Purpose zkSNARKs Without Trusted Setup. In *CRYPTO*.
- [53] Daniel Shanks. 1971. Class number, a theory of factorization, and genera. In *Proceedings of Symposia in Pure Mathematics*.
- [54] Shunya Tamai and Shoji Kasahara. 2024. DAO voting mechanism resistant to whale and collusion problems. *Frontiers Blockchain* 7 (2024).
- [55] Justin Thaler. 2022. Proofs, Arguments, and Zero-Knowledge. *Foundations and Trends in Privacy and Security* 4, 2-4 (2022), 117–660.
- [56] Riad S. Wahby, Ioanna Tzialla, Abhi Shelat, Justin Thaler, and Michael Walfish. 2018. Doubly-Efficient zkSNARKs Without Trusted Setup. In *S&P*. IEEE, 926–943.
- [57] Nan Wang and Sid Chi-Kin Chau. 2022. Flashproofs: Efficient Zero-Knowledge Arguments of Range and Polynomial Evaluation with Transparent Setup. In *ASIACRYPT*. Springer, 219–248.
- [58] Nan Wang, Sid Chi-Kin Chau, and Dongxi Liu. 2024. SwiftRange: A Short and Efficient Zero-Knowledge Range Argument For Confidential Transactions and More. In *S&P*. IEEE, 1832–1848.
- [59] E Glen Weyl. 2017. The robustness of quadratic voting. *Public choice* 172, 1 (2017), 75–107.
- [60] Tiancheng Xie, Jiaheng Zhang, Yupeng Zhang, Charalampos Papamanthou, and Dawn Song. 2019. Libra: Succinct Zero-Knowledge Proofs with Optimal Prover Computation. In *CRYPTO*. Springer, 733–764.
- [61] Yang Yang, Zhangshuang Guan, Zhiguo Wan, Jian Weng, HweeHwa Pang, and Robert Deng. 2021. PriScore: Blockchain-Based Self-Tallying Election System Supporting Score Voting. *IEEE Transactions on Information Forensics and Security* 16 (2021), 4705–4720.
- [62] Zongyang Zhang, Zibo Zhou, Weihai Li, and Hongyu Tao. 2021. An Optimized Inner Product Argument with More Application Scenarios. In *ICICS*. Springer, 341–357.
- [63] Zibo Zhou, Zongyang Zhang, Hongyu Tao, Tianyu Li, and Boyu Zhao. 2023. Efficient inner product arguments and their applications in range proofs. *IET Information Security* 17, 3 (2023), 485–504.

A Interactive zero-knowledge argument of knowledge

Definition A.1 (Interactive zero-knowledge argument of knowledge). An interactive zero-knowledge argument of knowledge (ZKAoK) for $\mathcal{R}$ is a tuple of PPT algorithms $\Gamma = (\text{Setup}, \mathcal{P}, \mathcal{V})$ with the following syntax:

- $\text{pp} \leftarrow \text{Setup}(1^\lambda)$: on input security parameter λ with the unary form, outputs public parameters pp .
- $tr \leftarrow \langle \mathcal{P}(w), \mathcal{V} \rangle(\text{pp}, x)$: an interactive protocol between a PPT prover $\mathcal{P}$ and verifier $\mathcal{V}$. Both $\mathcal{P}$ and $\mathcal{V}$ take public parameters pp and instance x as input. $\mathcal{P}$ additionally inputs a witness w . The protocol outputs a transcript tr . We write $\langle \mathcal{P}(w), \mathcal{V} \rangle(\text{pp}, x) = b$, where $b = 1$ if $\mathcal{V}$ accepts and $b = 0$ if $\mathcal{V}$ rejects. We call tr is accepting if $\mathcal{V}$ finally accepts.

We define the following properties for Γ .

Definition A.2 (Public coin). The verifier chooses all his messages uniformly at random.

Definition A.3 (Perfect completeness). Γ has perfect completeness if for all $(\text{pp}, x; w) \in \mathcal{R}$ where $\text{pp} \leftarrow \text{Setup}(1^\lambda)$, it holds that $\Pr[\langle \mathcal{P}(w), \mathcal{V} \rangle(\text{pp}, x) = 1] = 1$.

Definition A.4 (Computational witness-extended emulation [5]). Γ has computational witness-extended emulation if for all deterministic polynomial time $\mathcal{P}^*$, there exists an expected polynomial time emulator $\mathcal{E}$ such that for all non-uniform polynomial time

adversaries $\mathcal{A}$,

$$\begin{aligned} & \Pr \left[\mathcal{A}(tr) = 1 \mid \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda); (x, st) \leftarrow \mathcal{A}(\text{pp}); \\ tr \leftarrow \langle \mathcal{P}^*(st), \mathcal{V} \rangle(\text{pp}, x) \end{array} \right] - \\ & \Pr \left[\begin{array}{l} \mathcal{A}(tr) = 1 \text{ if } tr \text{ is accepting,} \\ \text{then } (\text{pp}, x; w) \in \mathcal{R} \end{array} \mid \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda); \\ (x, st) \leftarrow \mathcal{A}(\text{pp}); \\ (tr, w) \leftarrow \mathcal{E}^O(\text{pp}, x) \end{array} \right] \\ & \leq \text{negl}(\lambda), \end{aligned}$$

where $\mathcal{E}$ has access to an oracle $O = \langle \mathcal{P}^*(st), \mathcal{V} \rangle(\text{pp}, x)$ that can be rewound to any round and run again on fresh verifier randomness.

Definition A.5 (Perfect special honest-verifier zero-knowledge [5]). Γ has perfect special honest-verifier zero-knowledge if there exists probabilistic polynomial time simulator $\mathcal{S}$ such that for all adversaries $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$,

$$\begin{aligned} & \Pr \left[(\text{pp}, x; w) \in \mathcal{R} \wedge \mathcal{A}_2(tr) = 1 \mid \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda); \\ (x, w, \rho) \leftarrow \mathcal{A}_1(\text{pp}); \\ tr \leftarrow \langle \mathcal{P}(w), \mathcal{V}(\rho) \rangle(\text{pp}, x) \end{array} \right] \\ & = \Pr \left[(\text{pp}, x; w) \in \mathcal{R} \wedge \mathcal{A}_2(tr) = 1 \mid \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda); \\ (x, w, \rho) \leftarrow \mathcal{A}_1(\text{pp}); \\ tr \leftarrow \mathcal{S}(\text{pp}, x, \rho) \end{array} \right], \end{aligned}$$

where ρ is the randomness used by the verifier.

Compared to Definition A.4 and Definition A.5, we also consider two weaker properties: computational knowledge soundness and perfect honest-verifier zero-knowledge. Specifically, witness-extended emulation strengthens the knowledge-soundness definition so that the extractor outputs not only a witness but also a simulated transcript between the prover and verifier [6]. We define the two properties according to [8].

Definition A.6 (Computational knowledge soundness). Γ has computational knowledge soundness if there exists a PPT extractor $\mathcal{E}$ such that given oracle access to any pair of PPT adversarial prover algorithm $(\mathcal{A}_1, \mathcal{A}_2)$,

$$\Pr \left[\begin{array}{l} \langle \mathcal{A}_2(st), \mathcal{V} \rangle(\text{pp}, x) = 1 \\ \wedge \\ (\text{pp}, x; w) \notin \mathcal{R} \end{array} \mid \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda); \\ (x, st) \leftarrow \mathcal{A}_1(\text{pp}); \\ w \leftarrow \mathcal{E}^{\mathcal{A}_1, \mathcal{A}_2}(\text{pp}, x) \end{array} \right] \leq \text{negl}(\lambda).$$

Definition A.7 (Perfect honest-verifier zero-knowledge). Γ has perfect honest-verifier zero-knowledge if there exists a PPT simulator $\mathcal{S}$ such that for all adversaries $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ and auxiliary input $z \in \{0, 1\}^{\text{poly}(\lambda)}$,

$$\begin{aligned} & \Pr \left[\begin{array}{l} \langle \mathcal{P}(\text{pp}, w), \mathcal{A}_2(st) \rangle(x) = 1; \\ \wedge (\text{pp}, x; w) \in \mathcal{R} \end{array} \mid \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda); \\ (x, w, st) \leftarrow \mathcal{A}_1(z, \text{pp}) \end{array} \right] \\ & = \Pr \left[\begin{array}{l} \langle \mathcal{S}(\sigma, z, \text{pp}), \mathcal{A}_2(st) \rangle(x) = 1; \\ \wedge (\text{pp}, x; w) \in \mathcal{R} \end{array} \mid \begin{array}{l} (\text{pp}, \sigma) \leftarrow \mathcal{S}(1^\lambda); \\ (x, w, st) \leftarrow \mathcal{A}_1(z, \text{pp}) \end{array} \right]. \end{aligned}$$

B Commitment schemes

A commitment scheme should satisfy binding (see Definition B.1) and hiding (see Definition B.2) properties.

Definition B.1 (Computationally binding). A commitment scheme $\mathcal{C}$ is computationally binding if for all PPT adversaries $\mathcal{A}$,

$$\Pr \left[\begin{array}{l} b_0 = b_1 = 1 \wedge \\ m_0 \neq m_1 \end{array} \mid \begin{array}{l} ck \leftarrow \text{Setup}(1^\lambda); \\ (C, m_0, r_0, m_1, r_1) \leftarrow \mathcal{A}(ck); \\ b_0 \leftarrow \text{Open}(ck, C, m_0, r_0); \\ b_1 \leftarrow \text{Open}(ck, C, m_1, r_1) \end{array} \right] \leq \text{negl}(\lambda).$$

Definition B.2 (Perfectly hiding). A commitment scheme C is perfectly hiding if for every unbounded adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$,

$$\Pr \left[b = b' \mid \begin{array}{l} ck \leftarrow \text{Setup}(1^\lambda); \\ (m_0, m_1, st) \leftarrow \mathcal{A}_1(ck); b \xleftarrow{\$} \{0, 1\}; \\ (C_b, r_b) \leftarrow \text{Com}(ck, m_b); b' \leftarrow \mathcal{A}_2(ck, C_b, st) \end{array} \right] = \frac{1}{2}.$$

For Pedersen commitment scheme [44], the commitment key is $(p, \mathbb{G}, g, h)$, where h is a uniformly random group element. To commit a message $m \in \mathbb{F}_p$, the Com algorithm picks $r \xleftarrow{\$} \mathbb{F}_p$, computes $C := g^m h^r$, and outputs (C, r) .

The Pedersen vector commitment scheme is a natural variant of Pedersen commitment scheme. The commitment key is $(p, \mathbb{G}, g, h_1, \dots, h_n)$, where $h_1, \dots, h_n$ are uniformly random group elements. To commit a message $\mathbf{m} = (m_1, \dots, m_n) \in \mathbb{F}_p^n$, the Com algorithm picks $r \xleftarrow{\$} \mathbb{F}_p$, computes $C := g^r \cdot \prod_{i \in [n]} h_i^{m_i}$, and outputs (C, r) .

Pedersen commitment and Pedersen vector commitment schemes are computationally binding and perfectly hiding under the discrete logarithm assumption. Also, they are homomorphic. For $m_1, m_2 \in \mathbb{M}$, we have $\text{Com}(ck, m_1) \cdot \text{Com}(ck, m_2) = \text{Com}(ck, m_1 + m_2)$.

A polynomial over $\mathbb{F}$ is an expression consisting of a sum of monomials where each monomial is the product of a constant (from $\mathbb{F}$) and powers of one or more variables (which take values from $\mathbb{F}$). A multivariate polynomial is a polynomial with more than one variable. A multilinear polynomial is a multivariate polynomial where the degree in each variable is at most one.

C AND composition for proving knowledge of the discrete logarithm

We present a Σ protocol for the following $\mathcal{R}_{\text{dl}}$ relation by composing the Schnorr protocol.

$$\mathcal{R}_{\text{dl}} := \left\{ (n \in \mathbb{N}, \mathbf{h} \in \mathbb{G}^n, \mathbf{y} \in \mathbb{G}^n; \mathbf{x} \in \mathbb{F}_p^n) : y_j = h_j^{x_j} \text{ for } j \in [n] \right\}.$$

- $\mathcal{P}$ samples $r_1, \dots, r_n \xleftarrow{\$} \mathbb{F}_p$, computes and sends $\{R_j := h_j^{r_j}\}_{j \in [n]}$.
- $\mathcal{V}$ samples and sends $e \xleftarrow{\$} \mathbb{F}_p$.
- $\mathcal{P}$ computes and sends $\{z_j := r_j + e \cdot x_j\}_{j \in [n]}$.
- $\mathcal{V}$ checks $h_j^{z_j} = R_j \cdot y_j^e$ for $j \in [n]$. If these equations hold, $\mathcal{V}$ accepts; otherwise, $\mathcal{V}$ rejects.

D AND composition for proving the discrete logarithm equality

We present a Σ protocol for the following $\mathcal{R}_{\text{eq}}$ relation.

$$\mathcal{R}_{\text{eq}} := \left\{ (n \in \mathbb{N}, \mathbf{h}, \mathbf{g}, \mathbf{Y} \in \mathbb{G}^n, \mathbf{y}, \mathbf{B} \in \mathbb{G}^n; \mathbf{v}, \mathbf{x} \in \mathbb{F}_p^n) : \begin{array}{l} y_j = h_j^{x_j} \wedge B_j = g_j^{v_j} Y_j^{x_j} \text{ for } j \in [n] \end{array} \right\}.$$

- $\mathcal{P}$ samples $r_1, s_1, \dots, r_n, s_n \xleftarrow{\$} \mathbb{F}_p$, computes and sends $\{R_j := h_j^{r_j}, S_j := g_j^{s_j} Y_j^{r_j}\}_{j \in [n]}$.
- $\mathcal{V}$ samples and sends $e \xleftarrow{\$} \mathbb{F}_p$.
- $\mathcal{P}$ computes and sends $\{a_j := r_j + e \cdot x_j, b_j := s_j + e \cdot v_j\}_{j \in [n]}$.
- $\mathcal{V}$ checks $h_j^{a_j} = R_j \cdot y_j^e$ and $g_j^{b_j} Y_j^{a_j} = S_j \cdot B_j^e$ for $j \in [n]$. If these equations hold, $\mathcal{V}$ accepts; otherwise, $\mathcal{V}$ rejects.

E The sum-check protocol

The sum-check protocol satisfies completeness and soundness:

- **Completeness.** If $T = \sum_{b \in \{0,1\}^\ell} \tilde{g}(b)$, then for an honest $\mathcal{P}$ and for all $r \in \mathbb{F}^\ell$, $\Pr[c \leftarrow \langle \mathcal{P}, \mathcal{V}(r) \rangle (\tilde{g}, \ell, d, T) \wedge \tilde{g}(r) = c] = 1$.
- **Soundness.** If $T \neq \sum_{b \in \{0,1\}^\ell} \tilde{g}(b)$, then for any adversary $\mathcal{P}^*$ and for all $r \in \mathbb{F}^\ell$, $\Pr[c \leftarrow \langle \mathcal{P}^*, \mathcal{V}(r) \rangle (\tilde{g}, \ell, d, T) \wedge \tilde{g}(r) = c] \leq d\ell/|\mathbb{F}|$, where d is the maximal value in d .

We present the specification of $c \leftarrow \langle \mathcal{P}, \mathcal{V}(r) \rangle (\tilde{g}, \ell, d, T)$ used in Section 4 for proving $T = \sum_{b \in \{0,1\}^\ell} \tilde{g}(b)$, where $\tilde{g} : \mathbb{F}^\ell \rightarrow \mathbb{F}$ is an ℓ -variate polynomial with each variable X_i having a degree at most d_i for $i \in [\ell]$ and $r \in \mathbb{F}^\ell$ is the verifier's randomness.

This protocol consists of ℓ rounds [39, 55]. In each round, a prover sends a univariate polynomial to a verifier. The verifier performs some checks and then generates a random challenge.

- In Round 1, $\mathcal{P}$ sends the univariate polynomial

$$\tilde{g}_1(X_1) := \sum_{(b_2, \dots, b_\ell) \in \{0,1\}^{\ell-1}} \tilde{g}(X_1, b_2, \dots, b_\ell)$$

to $\mathcal{V}$. $\mathcal{V}$ checks that $\tilde{g}_1$ is a univariate polynomial of degree at most d_1 and $T = \tilde{g}_1(0) + \tilde{g}_1(1)$. $\mathcal{V}$ samples $r_1 \xleftarrow{\$} \mathbb{F}$ and sends it to $\mathcal{P}$.

- In Round i for $1 < i < \ell$, $\mathcal{P}$ sends the univariate polynomial

$$\tilde{g}_i(X_i) := \sum_{(b_{i+1}, \dots, b_\ell) \in \{0,1\}^{\ell-i}} \tilde{g}(r_1, \dots, r_{i-1}, X_i, b_{i+1}, \dots, b_\ell)$$

to $\mathcal{V}$. $\mathcal{V}$ checks that $\tilde{g}_i$ is a univariate polynomial of degree at most d_i and $\tilde{g}_{i-1}(r_{i-1}) = \tilde{g}_i(0) + \tilde{g}_i(1)$. $\mathcal{V}$ samples $r_i \xleftarrow{\$} \mathbb{F}$ and sends it to $\mathcal{P}$.

- In Round ℓ , $\mathcal{P}$ sends the univariate polynomial

$$\tilde{g}_\ell(X_\ell) := \tilde{g}(r_1, \dots, r_{\ell-1}, X_\ell)$$

to $\mathcal{V}$. $\mathcal{V}$ checks that $\tilde{g}_\ell$ is a univariate polynomial of degree at most d_ℓ and $\tilde{g}_{\ell-1}(r_{\ell-1}) = \tilde{g}_\ell(0) + \tilde{g}_\ell(1)$. $\mathcal{V}$ samples $r_\ell \xleftarrow{\$} \mathbb{F}$ and outputs a claimed evaluation $c := \tilde{g}(r_\ell)$, which should be equal to $\tilde{g}(r_1, \dots, r_\ell)$.

F A zero-knowledge Eval protocol of the discrete-logarithm-based multilinear PCS

We present the specification of the zero-knowledge Eval protocol used in Section 4. Formally, it is an interactive ZKAoK for the following linear relation:

$$\mathcal{R}_{\text{ln}} := \left\{ (n \in \mathbb{N}, \mathbf{g} \in \mathbb{G}^n, C \in \mathbb{G}, \mathbf{b} \in \mathbb{F}_p^n, t \in \mathbb{F}_p; \mathbf{a} \in \mathbb{F}_p^n) : \begin{array}{l} C = \mathbf{g}^{\mathbf{a}} \wedge t = \langle \mathbf{a}, \mathbf{b} \rangle \end{array} \right\},$$

which is exactly an instantiation of the linear relation in [1]. In the following, we present their compressed Σ protocol for proving $\mathcal{R}_{\text{ln}}$.

- $\mathcal{P}$ samples $\mathbf{r} \xleftarrow{\$} \mathbb{F}_p^n$. $\mathcal{P}$ then computes and sends $R := \mathbf{g}^{\mathbf{r}}, s := \langle \mathbf{r}, \mathbf{b} \rangle$.
- $\mathcal{V}$ samples and sends $e \xleftarrow{\$} \mathbb{F}_p$.
- $\mathcal{P}$ computes $\mathbf{z} := \mathbf{r} + e \cdot \mathbf{a}$.
- $\mathcal{V}$ samples and sends $c \xleftarrow{\$} \mathbb{F}_p$.

- Let $q \in \mathbb{G}$ be a uniformly random element. $\mathcal{P}$ and $\mathcal{V}$ run the following process to prove the relation defined below on input $(n, \mathbf{g}, q^c, R \cdot C^e \cdot (q^c)^{s+et}, \mathbf{b}; \mathbf{z})$.

$$\{(n \in \mathbb{N}, \mathbf{g} \in \mathbb{G}^n, u \in \mathbb{G}, P \in \mathbb{G}, \mathbf{b} \in \mathbb{F}_p^n, \mathbf{z} \in \mathbb{F}_p^n) : P = \mathbf{g}^{\mathbf{z}} u^{\langle \mathbf{z}, \mathbf{b} \rangle}\}.$$
 1. Split the vectors as $\mathbf{g} = (\mathbf{g}_\ell, \mathbf{g}_r), \mathbf{z} = (\mathbf{z}_\ell, \mathbf{z}_r), \mathbf{b} = (\mathbf{b}_\ell, \mathbf{b}_r)$.
 2. $\mathcal{P}$ computes and sends $A := \mathbf{g}_r^{\mathbf{z}_\ell} \cdot u^{\langle \mathbf{z}_\ell, \mathbf{b}_r \rangle}, B := \mathbf{g}_\ell^{\mathbf{z}_r} \cdot u^{\langle \mathbf{z}_r, \mathbf{b}_\ell \rangle}$.
 3. $\mathcal{V}$ samples and sends $e \xleftarrow{\$} \mathbb{F}_p$. $\mathcal{V}$ then computes $P' := AP^e B^{e^2}, \mathbf{g}' := \mathbf{g}_\ell^e \circ \mathbf{g}_r, \mathbf{b}' := e \cdot \mathbf{b}_\ell + \mathbf{b}_r$.
 4. $\mathcal{P}$ computes $\mathbf{z}' := \mathbf{z}_\ell + e \cdot \mathbf{z}_r, \mathbf{g}' := \mathbf{g}_\ell^e \circ \mathbf{g}_r, \mathbf{b}' := e \cdot \mathbf{b}_\ell + \mathbf{b}_r$.
 5. If $\mathbf{z}' \in \mathbb{F}_p^4$:
 - $\mathcal{P}$ sends $\mathbf{z}'$.
 - $\mathcal{V}$ checks $P' = \mathbf{g}'^{\mathbf{z}'} u^{\langle \mathbf{z}', \mathbf{b}' \rangle}$. If this equation holds, $\mathcal{V}$ accepts; otherwise, $\mathcal{V}$ rejects.
- Else:
 - $\mathcal{P}$ and $\mathcal{V}$ recursively repeat the above process from Step 1 with updated input $(n/2, \mathbf{g}', u, P', \mathbf{b}'; \mathbf{z}')$.

G Proof of Theorem 4.1

THEOREM 4.1. *The protocol described in Section 4.2 is an interactive ZKAoK for relation $\mathcal{R}_{ss}$ under the discrete logarithm assumption. It has perfect completeness, computational knowledge soundness and perfect honest-verifier zero-knowledge.*

PROOF. We now describe the proof of Theorem 4.1.

Perfect completeness. Suppose that $(n, \mathbf{g}, Y, g, h, B, \tilde{B}; v, \mathbf{x}, u, r) \in \mathcal{R}_{ss}$. By the completeness of the Σ protocol, the check $\tilde{B} \cdot \tilde{C}^\rho = \mathbf{g}^{\mathbf{z}} h^{\mathbf{z}'}$ will pass. By construction, $\mathbf{z} = \sum_{b \in \{0,1\}^\ell} \tilde{f}^*(b) \cdot \tilde{f}^*(b) + \rho \cdot \tilde{g}^*(b)$. Then by the completeness of the sum-check protocol and PCS, the verifier always accepts. We conclude that the protocol has perfect completeness.

Computational knowledge soundness. We show how to construct an extractor to extract a valid witness for relation $\mathcal{R}_{ss}$. Since the PCS is extractable under the discrete logarithm assumption, there exists an extractor $\mathcal{E}_{PCS}$ to extract polynomials $\tilde{f}^*, \tilde{g}^*$ such that $B \cdot C_R^{\tilde{Z}(r)} = \text{PCom}(\tilde{f}^*), v_{f^*} = \tilde{f}^*(r), C_{g^*} = \text{PCom}(\tilde{g}^*), v_{g^*} = \tilde{g}^*(r)$ and $\tilde{f}^*(r) = \tilde{f}(r) + \tilde{R}(r) \cdot \tilde{Z}(r)$, where $\tilde{f}, \tilde{R}$ are openings of B, C_R , respectively. By repeating this process with two different challenges r, r' and manipulating the equation $B \cdot C_R^{\tilde{Z}(r)} = \text{PCom}(\tilde{f}^*)$ to eliminate C_R , we could extract $(v, \mathbf{x})$ such that $B = \mathbf{g}^v Y^{\mathbf{x}}$.

By the soundness of the sum-check protocol, we have that $\mathbf{z} = \sum_{b \in \{0,1\}^\ell} \tilde{f}^*(b) \cdot \tilde{f}^*(b) + \rho \cdot \tilde{g}^*(b)$ with overwhelming probability, which implies that $\mathbf{z} = \sum_{b \in \{0,1\}^\ell} \tilde{f}(b) \cdot \tilde{f}(b) + \rho \cdot \tilde{g}^*(b) = \langle v, v \rangle + \rho \cdot \sum_{b \in \{0,1\}^\ell} \tilde{g}^*(b)$ due to the form of $\tilde{f}^*$. Since the Σ protocol is special sound, there exists an extractor $\mathcal{E}_\Sigma$ to extract (u, r, u^*, r^*) such that $\hat{B} = g^u h^r, \hat{C} = g^{u^*} h^{r^*}$. Then for two different challenges ρ_1, ρ_2 , we have $u + \rho_1 u^* = z_1 = \langle v, v \rangle + \rho_1 \cdot \sum_{b \in \{0,1\}^\ell} \tilde{g}^*(b)$ and $u + \rho_2 u^* = z_2 = \langle v, v \rangle + \rho_2 \cdot \sum_{b \in \{0,1\}^\ell} \tilde{g}^*(b)$, which implies that $u = \langle v, v \rangle$. Thus, $(v, \mathbf{x}, u, r)$ is a valid witness for $\mathcal{R}_{ss}$. We conclude that the protocol has computational knowledge soundness.

Perfect honest-verifier zero-knowledge. We show that the proof reveals nothing about the witness. By the hiding property of the PCS, elements C_{g^*}, C_R are uniformly random. By the perfect special

honest-verifier zero-knowledge of the Σ protocol, the process for proving the knowledge of (u, r) such that $\hat{B} = g^u h^r$ reveals nothing about the witness. According to [60], the sum-check protocol retains perfect honest-verifier zero-knowledge. Since we use a random polynomial $\tilde{R}$ to mask $\tilde{f}$, the evaluation v_{f^*} is uniformly random. Then according to [1], the Eval protocol of the PCS reveals nothing about the witness. We conclude that the protocol has perfect honest-verifier zero-knowledge. $\square$

H AND composition for proving knowledge of Pedersen commitment openings

We present a Σ protocol for the following $\mathcal{R}_{pd}$ relation.

$$\mathcal{R}_{pd} := \left\{ (n \in \mathbb{N}, \mathbf{g}, \mathbf{h} \in \mathbb{G}^n, \mathbf{B} \in \mathbb{G}^n; v, \mathbf{x} \in \mathbb{F}_p^n) : \begin{array}{l} B_j = g_j^{v_j} h_j^{x_j} \text{ for } j \in [n] \end{array} \right\}.$$

- $\mathcal{P}$ samples $r_1, s_1, \dots, r_n, s_n \xleftarrow{\$} \mathbb{F}_p$, computes and sends $\{R_j := g_j^{r_j} h_j^{s_j}\}_{j \in [n]}$.
- $\mathcal{V}$ samples and sends $e \xleftarrow{\$} \mathbb{F}_p$.
- $\mathcal{P}$ computes and sends $\{a_j := r_j + e \cdot v_j, b_j := s_j + e \cdot x_j\}_{j \in [n]}$.
- $\mathcal{V}$ checks $g_j^{a_j} h_j^{b_j} = R_j \cdot B_j^e$ for $j \in [n]$. If these equations hold, $\mathcal{V}$ accepts; otherwise, $\mathcal{V}$ rejects.

I Proof of Theorem 5.2

THEOREM 5.2. *The protocol described in Section 5.4 is an interactive ZKAoK for relation $\mathcal{R}'_{ar}$ under the discrete logarithm assumption. It has perfect completeness, computational witness-extended emulation and perfect special honest-verifier zero-knowledge.*

PROOF. We now describe the proof of Theorem 5.2.

Perfect completeness. If $(v, \hat{v}, \mathbf{x}, \hat{\mathbf{x}})$ is a valid witness for $\mathcal{R}'_{ar}$, then we have for a random $y, t_0 = 0$. When $nb + \bar{b} \leq 21$, by the homomorphic property of the Pedersen commitment and Pedersen vector commitment, the verifier always accepts. When $nb + \bar{b} > 21$, additionally by the perfect completeness of the IPA protocol, the verifier always accepts. We conclude that the protocol has perfect completeness.

Computational witness-extended emulation. According to the forking lemma of [4, 5], we need to construct an extractor $\mathcal{E}$ to extract a valid witness for $\mathcal{R}'_{ar}$ in probabilistic polynomial time with non-negligible probability, using accepting transcripts of size polynomial in λ .

The extractor $\mathcal{E}$ rewinds the prover with $4(nb + \bar{b}) + 2n + 2$ different values of challenge y and 17 different values of challenge x , and obtains $(4(nb + \bar{b}) + 2n + 2) \cdot 17$ accepting transcripts. For each transcript, $\mathcal{E}$ could extract L directly (when $nb + \bar{b} \leq 21$) or by invoking the extractor for the IPA protocol (when $nb + \bar{b} > 21$), such that

$$\mathbf{g}^L = A^x \cdot B^{x^{-1}} \cdot \hat{A}^{y^{nb+\bar{b}}} \cdot x^2 \cdot \hat{B}^{x^{-2}} \cdot \left(\prod_{j \in [n+1]} C_j \right)^{x^3} \cdot \left(\prod_{j \in [n+1]} \hat{C}_j \right)^{x^4} \cdot D^{x^5} \cdot h^{-\rho}. \quad (2)$$

For fixed challenge y and 7 different challenges x , we have 7 accepting transcripts that satisfy Equation (2). By taking appropriate linear combinations of these equations, we could compute

$\mathbf{a}', \rho'_a, \hat{\mathbf{a}}', \rho'_{\hat{a}}, \mathbf{b}', \rho'_b, \hat{\mathbf{b}}', \rho'_{\hat{b}}, \mathbf{c}', \rho'_c, \hat{\mathbf{c}}', \rho'_{\hat{c}}, \mathbf{d}', \rho'_d$ such that

$$A = \mathbf{g}^{\mathbf{a}'} h^{\rho'_a}, \hat{A} = \mathbf{g}^{\hat{\mathbf{a}}'} h^{\rho'_{\hat{a}}}, B = \mathbf{g}^{\mathbf{b}'} h^{\rho'_b}, \hat{B} = \mathbf{g}^{\hat{\mathbf{b}}'} h^{\rho'_{\hat{b}}}, D = \mathbf{g}^{\mathbf{d}'} h^{\rho'_d},$$

$$\prod_{j \in [n+1]} C_j = \mathbf{g}^{\mathbf{c}'} h^{\rho'_c}, \prod_{j \in [n+1]} \hat{C}_j = \mathbf{g}^{\hat{\mathbf{c}}'} h^{\rho'_{\hat{c}}}.$$

According to the discrete logarithm assumption, these computed values are independent of y, x , and

$$L = \mathbf{a}' \cdot x + \mathbf{b}' \cdot x^{-1} + y^{nb+\bar{b}} \hat{\mathbf{a}}' \cdot x^2 + \hat{\mathbf{b}}' \cdot x^{-2} + \mathbf{c}' \cdot x^3 + \hat{\mathbf{c}}' \cdot x^4 + \mathbf{d}' \cdot x^5.$$

Since we have additionally proved that $\{C_j, \hat{C}_j\}_{j \in [n+1]}$ are well-formed, we could extract $\mathbf{v}', \hat{\mathbf{v}}', \mathbf{x}', \hat{\mathbf{x}}' \in \mathbb{F}_p^{n+1}$ from $\mathbf{c}', \hat{\mathbf{c}}'$ such that for $j \in [n+1]$, $C_j = g_j^{\mathbf{v}'_j} \cdot h_j^{\mathbf{x}'_j}$ and $\hat{C}_j = g_j^{\hat{\mathbf{v}}'_j} \cdot h_j^{\hat{\mathbf{x}}'_j}$.

For fixed challenge y and 17 different challenges x , we have 17 accepting transcripts that satisfy $g_1^{\hat{\mathbf{t}}} h^{\tau} = \prod_{i=-6, i \neq 0}^{10} T_i^{x^i}$. By taking appropriate linear combinations of any 16 equations, we could compute $\{\hat{t}'_i, \tau'_i\}_{i=-6, i \neq 0}^{10}$ such that $T_i = g_1^{\hat{t}'_i} h^{\tau'_i}$. According to the discrete logarithm assumption, these computed values are independent of x , and $\hat{t} = \sum_{i=-6, i \neq 0}^{10} \hat{t}'_i \cdot x^i$, where $\hat{t} = M(x)$. Since this equation holds for 17 different values of x , we have t_0 of $M(X)$ must be zero, which implies that $P(y) = 0$. Since the degree of $P(Y)$ is $4(nb + \bar{b}) + 2n + 1$ and $P(y) = 0$ for $4(nb + \bar{b}) + 2n + 2$ different values of y , we have the coefficients of $P(Y)$ must be zero. Then according to the constraint transformation, we have that $\mathbf{v}'_j, \hat{\mathbf{v}}'_j \in [0, 2^b]$ for $j \in [n]$ and $\mathbf{v}'_{n+1}, \hat{\mathbf{v}}'_{n+1} \in [0, 2^{\bar{b}}]$. Thus, $(\mathbf{v}', \hat{\mathbf{v}}', \mathbf{x}', \hat{\mathbf{x}}')$ is a valid witness for $\mathcal{R}'_{ar}$. By the discrete logarithm assumption, $\mathcal{E}$ fails with at most negligible probability.

The extractor $\mathcal{E}$ extracts a witness in probabilistic polynomial time with non-negligible probability, using accepting transcripts of size $(4(nb + \bar{b}) + 2n + 2) \cdot 17 \cdot \text{poly}(nb + \bar{b})$ that is polynomial in λ . We conclude that the protocol has computational witness-extended emulation.

Perfect special honest-verifier zero-knowledge. We construct a PPT simulator $\mathcal{S}$ that produces a transcript perfectly indistinguishable from a transcript of a real execution, given $(n, b, \bar{b} \in \mathbb{N}, \mathbf{g}, \mathbf{h} \in \mathbb{G}^{n+1}, C, \hat{C} \in \mathbb{G}^{n+1}, \mathbf{h} \in \mathbb{G}, \{\mathbf{u}_j \in \mathbb{G}^{b-2}\}_{j \in [n]}, \mathbf{u}_{n+1} \in \mathbb{G}^{\bar{b}-2})$ and the verifier's randomness. The simulator $\mathcal{S}$ acts as follows:

– Compute y, x using the verifier's randomness.

– Sample $A, \hat{A}, B, \hat{B}, \{T_i\}_{i=-6, i \neq 0}^9 \xleftarrow{\$} \mathbb{G}$.

– Sample $L \xleftarrow{\$} \mathbb{F}_p^{nb+\bar{b}}, \tau, \rho \xleftarrow{\$} \mathbb{F}_p$.

– Compute $R := R(x), \hat{t} := M(x)$,

$$D := \left(\mathbf{g}^L \cdot A^{-x} B^{-x^{-1}} \hat{A}^{-y^{nb+\bar{b}}} \hat{B}^{-x^{-2}} \left(\prod_{j \in [n+1]} C_j \right)^{-x^3} \left(\prod_{j \in [n+1]} \hat{C}_j \right)^{-x^4} \cdot h^{\rho} \right)^{x^{-5}},$$

$$T_{10} := \left(g_1^{\hat{t}} h^{\tau} \cdot \prod_{i=-6, i \neq 0}^9 T_i^{-x^i} \right)^{x^{-10}}.$$

– If $nb + \bar{b} \leq 21$, output simulated transcript $(A, \hat{A}, B, \hat{B}, D, y, \{T_i\}_{i=-6, i \neq 0}^{10}, x, L, \tau, \rho)$.

– Else:

• Compute $P := \mathbf{g}^L, Q := P \cdot \mathbf{h}^{2R}$.

• Run the IPA protocol on input $(\mathbf{g}, \mathbf{h}, P, Q, \hat{t} + 2 \cdot \langle \mathbf{1}_{nb+\bar{b}}, (y^{2(nb+\bar{b}}) + y^{3(nb+\bar{b}}) \cdot \mathbf{y}^{nb+\bar{b}}) \rangle; L, L \circ \mathbf{y}^{nb+\bar{b}} + 2R)$, and obtain the protocol's accepting transcript trc .

• Output simulated transcript $(A, \hat{A}, B, \hat{B}, D, y, \{T_i\}_{i=-6, i \neq 0}^{10}, x, \hat{t}, \tau, \rho, trc)$.

We now show that the simulated transcript has the same distribution as a real transcript. Since the Pedersen commitment and Pedersen vector commitment are perfectly hiding, $(A, \hat{A}, B, \hat{B}, \{T_i\}_{i=-6, i \neq 0}^9)$ in the simulated and real transcripts are all uniformly distributed. In both transcripts, y, x are chosen independently and randomly, and L, τ, ρ are uniformly distributed. Given these elements, D, T_{10} are uniquely defined by the verification equations. When using the IPA, since the simulated witness $(L, L \circ \mathbf{y}^{nb+\bar{b}} + 2R)$ is valid and identically distributed to a real witness, the transcript trc is identically distributed to a real one. Therefore, the simulated transcript is perfectly indistinguishable from a real transcript. The simulator $\mathcal{S}$ runs in time $O(\mathcal{P} + \mathcal{V})$ and is thus efficient. We conclude that the protocol has perfect special honest-verifier zero-knowledge. $\square$